BASE DE DATOS PARA RESERVAS DE HOTELES



Grado en Inteligencia Artificial
3º curso

Paula Esmorís Rodríguez
María Gómez Fernández
Alejandra Ares Rivas

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA
ETSE Escola Técnica Superior de Enxeñaría

ÍNDICE

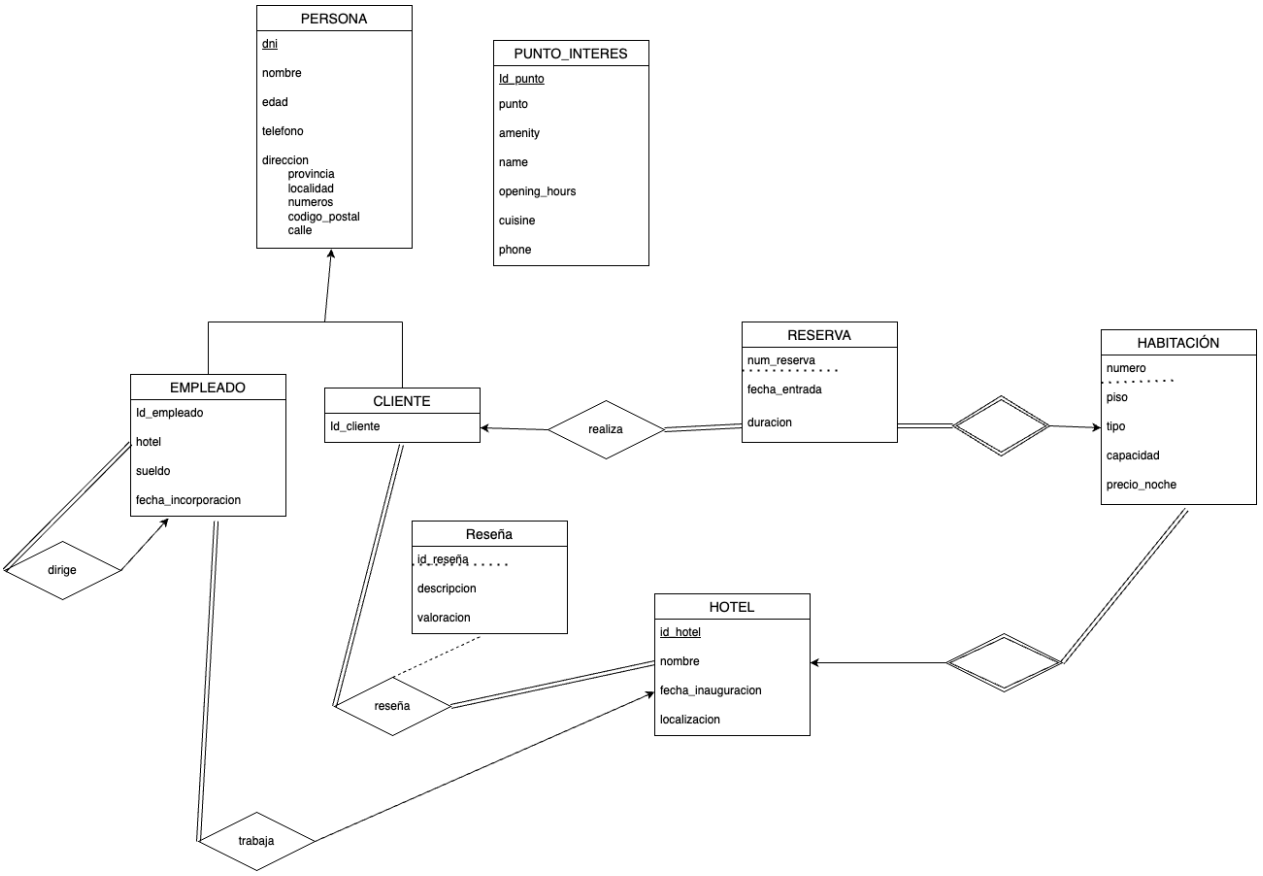
1. **Introducción y definición del alcance del proyecto** **3**
2. **Modelo Entidad - Relación** **4**
3. **Modelo relacional** **5**
4. **Modelo relacional extendido** **10**
    a. Modelo de tipos compuestos 10
    b. Modelo JSON 15
5. **Base de datos relacional distribuida** **20**
6. **Base de datos NoSQL documental: MongoDB** **22**
7. **Base de datos NoSQL de grafos: Neo4J** **30**
8. **Base de datos NoSQL de tipo Column Family: HBase** **36**

INTRODUCCIÓN Y DEFINICIÓN DEL ALCANCE DEL PROYECTO

Este proyecto se centra en la creación de una base de datos para un complejo de hoteles localizado en Santiago de Compostela, en la que podremos obtener información de cada uno de los hoteles que lo forman. Disponemos de información acerca de los clientes y de los trabajadores de los hoteles. Además, en nuestra base de datos guardamos información sobre cada reserva hecha por cada cliente con información adicional sobre la capacidad de la habitación, el tipo o el precio, entre otros. También se encuentran registradas cada reseña hecha por un cliente sobre una reserva en uno de los hoteles. En la información sobre los hoteles tenemos datos geométricos sobre la localización de cada hotel y a mayores tenemos información sobre los puntos de interés de la ciudad, para poder hacer consultas geométricas en base a la localización de cada hotel.

A continuación, abordaremos numerosas bases de datos, tanto SQL como NoSQL. Por una parte, crearemos bases relacionales, agregadas y de JSON. Por otra parte, trataremos otro tipo de bases de datos NoSQL, tratando bases de datos distribuidas, paralelas (con MongoDB), con grafos (Neo4J) y bases de datos de tipo Column Family (HBase). La intención de este proyecto es reflejar las numerosas formas que podemos tener de almacenar los datos y acceder a ellos mediante las consultas y transacciones, valorando asimismo las ventajas e inconvenientes de cada una de las implementaciones.

MODELO ENTIDAD - RELACIÓN



MODELO RELACIONAL

- **Persona** (dni, nombre, provincia, localidad, calle, numero, codigo_postal)
  - CP: dni

- **Empleado** (dni_empleado, hotel, id_empleado, cargo, sueldo, fecha_incorporacion, dni_director)
  - CP: dni_empleado
  - CF:
    - dni_empleado → Persona
    - dni_director → Empleado
    - hotel → hotel

- **Cliente**(dni_cliente, id_cliente)
  - CP:dni_cliente
  - CF: dni_cliente → Persona

- **Hotel**(id_hotel, nombre, fecha_inauguracion, localizacion)
  - CP: id_hotel

- **Habitacion**(hotel, numero, piso, tipo, capacidad, precio_noche)
  - CP:hotel, numero
  - CF: hotel → hotel

- **Reserva**(num_reserva, habitacion, hotel, nombre, fecha_entrada, duracion, cliente)
  - CP: num_reserva, numero, id_hotel
  - CF:
    - (habitacion, hotel) → habitacion
    - cliente → cliente

- **Cliente_reseña_hotel**(cliente, hotel, id_reseña, descripcion, valoracion)
  - CP:(cliente, hotel, id_reseña)
  - CF:
    - cliente → cliente
    - hotel → hotel

- **Punto_interes**(id_punto, punto, amenity, name, opening_hours, phone)

CONSULTAS SQL

Consulta 1: El personal del servicio de habitaciones se ha encontrado una lámpara rota en el pasillo de la planta 5 del hotel Araguaney y no saben qué cliente fue, solo saben que ha estado alojado en esta planta y que el año pasado esa lámpara no estaba rota. Queremos saber el nombre y dni de esa persona así como la habitación en la que se alojó y durante cuántos días.

```
select p.nombre, p.dni, h.numero as num_habitacion, r.duracion as dias_alojamiento
from personas p join clientes c on (p.dni = c.dni_cliente)
    join reservas r on (c.dni_cliente = r.cliente)
    join habitaciones h on (r.habitacion = h.numero and r.hotel = h.hotel)
where h.hotel = 1 and h.piso = 5 and extract(year from r.fecha_entrada) = 2025
```

	nombre	dni	num_habitacion	dias_alojamiento
1	Vicente Pérez	48783419F	17	14
2	Sergio Fernández	32212400K	105	2
3	Vicente Campos	56608488G	204	2

Consulta 2: Queremos hacer unas estadísticas sobre los ingresos obtenidos por las reservas de habitaciones y sobre los gastos que nos han supuesto el mantenimiento de todo el personal del “Hostal dos Reis Católicos” en los últimos 15 años.

```
with ingresos_anuales as(
    select extract(year from r.fecha_entrada) as ano,
    sum (r.duracion*h.precio_noche) as ingresos
    from reservas as r join habitaciones as h on (r.habitacion = h.numero and r.hotel = h.hotel)
    where r.hotel = 2
    group by ano
    order by ano desc
),
gastos_personal as(
    select ia.ano, sum(e.sueldo *12) as total_gastado
    from ingresos_anuales as ia, empleados as e
    where e.hotel = 2 and extract(year from e. fecha_incorporacion) <= ia.ano
    group by ia.ano
)
select
    i.ano,
    i.ingresos,
    g.total_gastado
from ingresos_anuales i left join gastos_personal g on (i.ano = g.ano)
order by i.ano desc
```

	ano	ingresos	total_gastado
1	2.025	62.539	570.312
2	2.024	26.754	570.312
3	2.023	38.355	535.380
4	2.022	18.024	500.100
5	2.021	22.132	449.160
6	2.020	30.546	408.396
7	2.019	34.649	392.196
8	2.018	49.691	380.016
9	2.017	40.234	362.604
10	2.016	29.662	346.692
11	2.015	52.908	309.264
12	2.014	33.729	309.264
13	2.013	52.636	225.984
14	2.012	36.874	169.236
15	2.011	31.151	132.024

Consulta 3: Queremos analizar cómo de adecuados son nuestros hoteles para familias con bebés, por lo que buscaremos aquellas reseñas que hagan referencia a bebés y mostraremos en la consulta qué persona puso la reseña, la valoración, el id de la reseña y lo que puso en ella.

```
SELECT p.nombre, r.valoracion, r.id_reseña, r.descripcion
FROM reseñas r join clientes c on (r.cliente = c.dni_cliente) join personas p on (p.dni = c.dni_cliente)
WHERE to_tsvector('english', descripcion) @@ to_tsquery('english', 'baby')
ORDER BY valoracion desc
```

	nombre	valoracion	id_reseña	descripcion
1	Nicolás Méndez	5	373	superb not asked, stayed mayfair 6 days 1 bedroom suite perfect, travelir
2	Clara Díaz	5	28	excellente resort resort excellent, went honeymoon just got, multiple rest
3	Pedro Fernández	5	242	great sejour husband sojourned loved, trying hotels paris finally perfect,
4	David Blanco	5	48	wana, england baby, woop woop, stayed hotel late august early sept fam
5	David Santos	4	239	loved place winter wife 1 year old child, loved place room better cuba sa
6	Vicente Pérez	1	626	not worth price date stay november 16-19 2008my family arrived hotel c
7	David Díaz	1	269	horrific despite friendly assistance time booking nothing prepared, book
8	Nicolás Ortega	1	423	not book hotel 5-6 months, arrived decribed building site, picked jaws gr

Consulta 4: Queremos hacer una lista de los clientes vips de cada hotel. Para eso consideramos a clientes que, o estuvieran en más de 5 estancias, o que dejaron más de 5 reseñas.

	hotel	cliente	dni_cliente
1	Hostal dos Reis Católicos	Alejandra Soto	76184458D
2	Hostal dos Reis Católicos	Iván Vega	94415651E
3	Hostal dos Reis Católicos	Jesús García	68675158D
4	Hotel Puerta del Camino	Alfonso Blanco	13246617X
5	Hotel Puerta del Camino	Cristina Díaz	86706296A
6	Hotel Puerta del Camino	Juan Sánchez	77368169P
7	Hotel Puerta del Camino	Rodrigo Blanco	27482426R

```
SELECT
    h.nombre AS hotel,
    p.nombre AS cliente,
    c.dni_cliente
    --COUNT(*) AS total_reservas
    --0 AS total_resenas
FROM reservas r
JOIN hoteles h ON r.hotel = h.id
JOIN clientes c ON r.cliente = c.dni_cliente
JOIN personas p ON c.dni_cliente = p.dni
GROUP BY h.nombre, p.nombre, c.dni_cliente
HAVING COUNT(*) > 5

UNION

SELECT
    h.nombre AS hotel,
    p.nombre AS cliente,
    c.dni_cliente
    --0 AS total_reservas,
    --COUNT(re.id_resena) AS total_resenas
FROM resennas re
JOIN hoteles h ON re.hotel = h.id
JOIN clientes c ON re.cliente = c.dni_cliente
JOIN personas p ON c.dni_cliente = p.dni
GROUP BY h.nombre, p.nombre, c.dni_cliente
HAVING COUNT(*) > 5
```

Consulta 5: Queremos hacer un estudio de las mejores habitaciones de cada hotel para poder reservarlas para clientes VIP en algún momento. Para eso hacemos un ranking de las 3 mejores habitaciones de cada hotel basándose en la media de la valoración y la cantidad total del número de reservas.

```sql
select *
from (
    select
        h.hotel as id_hotel,
        ht.nombre as nombre_hotel,
        h.numero as num_habitacion,
        h.tipo,
        h.capacidad,
        AVG(rs.valoracion) as media_valoracion,
        RANK() over (
            partition by h.hotel
            order by COUNT(r.num_reserva) desc, AVG(rs.valoracion) desc
        ) as ranking
    from hoteles.habitaciones h
    left join hoteles.reservas r
        on h.hotel = r.hotel and h.numero = r.habitacion
    left join hoteles.resennas rs
        on h.hotel = rs.hotel
    left join hoteles.hoteles ht
        on h.hotel = ht.id
    group by h.hotel, ht.nombre, h.numero, h.tipo, h.capacidad
) as ranked
where ranking <= 3
order by id_hotel, ranking;
```

select * from (select h.hotel as id_hotel, ht.nombre as r | Enter a SQL expression to filter results (use Ctrl+Space)

Grilla | Texto

	id_hotel	nombre_hotel	num_habitacion	tipo	capacidad	media_valoracion	ranking
1	1	Hotel Araguaney	143	Doble	2	3,9897435897	1
2	1	Hotel Araguaney	260	Familiar	4	3,9897435897	1
3	1	Hotel Araguaney	17	Individual	1	3,9897435897	3
4	1	Hotel Araguaney	68	Doble	2	3,9897435897	3
5	1	Hotel Araguaney	53	Doble	2	3,9897435897	3
6	1	Hotel Araguaney	47	Doble	2	3,9897435897	3
7	2	Hostal dos Reis Católicos	101	Doble	2	3,936329588	1
8	2	Hostal dos Reis Católicos	269	Familiar	4	3,936329588	1
9	2	Hostal dos Reis Católicos	192	Doble	2	3,936329588	3
10	2	Hostal dos Reis Católicos	237	Suite	2	3,936329588	3
11	2	Hostal dos Reis Católicos	45	Doble	2	3,936329588	3
12	2	Hostal dos Reis Católicos	151	Doble	2	3,936329588	3
13	2	Hostal dos Reis Católicos	308	Doble	2	3,936329588	3
14	3	Hotel Puerta del Camino	152	Doble	2	3,9582089552	1
15	3	Hotel Puerta del Camino	132	Doble	2	3,9582089552	2
16	3	Hotel Puerta del Camino	460	Doble	2	3,9582089552	2

Consulta 6: Ver cuántos empleados no tienen a nadie de subordinado (no son jefes) y mirar el dni del jefe directo y su cargo.

```sql
with recursive empleados_sin_subordinados as (
    select *
    from Empleado e
    where e.dni_empleado not in (
        select distinct dni_director
        from Empleado
        where dni_director is not null
    )
),
jerarquia as (
    select
        e.dni_empleado as empleado,
        e.cargo as cargo_empleado,
        e.dni_director as dni_jefe,
        1 as nivel
    from empleados_sin_subordinados e

    union all

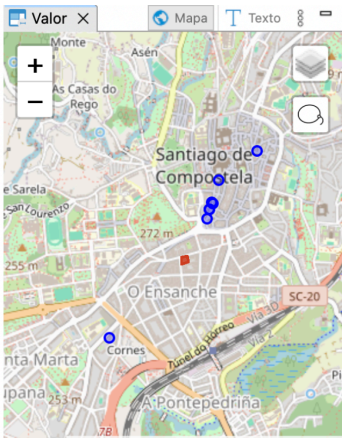
    select
        j.empleado,
        j.cargo_empleado,
        e.dni_director AS dni_jefe,
        j.nivel + 1
    from jerarquia j
    join Empleado e on j.dni_jefe = e.dni_empleado
    where j.dni_jefe is not null
)
-- Seleccionamos solo el jefe directo
select
    j.empleado,
    j.cargo_empleado,
    j.dni_jefe,
    e.cargo as cargo_jefe
from jerarquia j
left join Empleado e on j.dni_jefe = e.dni_empleado
where j.nivel = 1
order by j.empleado;
```

	empleado	cargo_empleado	dni_jefe	cargo_jefe
1	00651254V	Recepcionista	56144992N	Jefe de Recepción
2	02772688J	Técnico de Mantenimiento	06263162D	Jefe de Mantenimiento
3	02841500V	Técnico de Mantenimiento	42292514H	Jefe de Mantenimiento
4	02999244L	Botones	72009195L	Jefe de Recepción
5	04391822Y	Técnico de Mantenimiento	42292514H	Jefe de Mantenimiento
6	06011941A	Recepcionista	77054112P	Jefe de Recepción
7	06382995S	Recepcionista	77054112P	Jefe de Recepción
8	10527053A	Limpiador/a	45988552N	Jefa de Limpieza
9	10904380S	Auxiliar de Lavandería	45988552N	Jefa de Limpieza
10	11939686X	Recepcionista	72009195L	Jefe de Recepción
11	12670662Z	Técnico de Mantenimiento	42292514H	Jefe de Mantenimiento
12	14353053N	Botones	56144992N	Jefe de Recepción
13	17107946G	Técnico de Mantenimiento	33916872W	Jefe de Mantenimiento
14	17577868Y	Cocinero	02989767E	Jefe de Cocina
15	18798253L	Técnico de Mantenimiento	33916872W	Jefe de Mantenimiento
16	19528291J	Cocinero	45526763F	Jefe de Cocina
17	21193442Q	Técnico de Mantenimiento	33916872W	Jefe de Mantenimiento
18	21336239A	Cocinero	07810360J	Jefe de Cocina
19	22471050X	Botones	56144992N	Jefe de Recepción
20	22874771N	Botones	72009195L	Jefe de Recepción
21	23614865P	Camarero	02989767E	Jefe de Cocina
22	23969660F	Auxiliar de Lavandería	45988552N	Jefa de Limpieza
23	29821759D	Técnico de Mantenimiento	33916872W	Jefe de Mantenimiento

Consulta 7: Un cliente del Araguaney (id = 1) quiere saber qué opciones tiene para cenar esta noche. Específicamente, está buscando restaurantes de cocina regional que estén lo suficientemente cerca como para ir andando, así que quiere una lista de los que están a menos de 1 kilómetro (1000 metros) de distancia, ordenados de más cerca a más lejos, y obtener sus horas de apertura si es posible.

```
--Consulta 6:
select distinct pi.name, pi.opening_hours, st_distance(h.localizacion, pi.geom) AS distancia_en_metros, pi.geom, h.localizacion AS geom_hotel
from hoteles h, puntos_interes pi
where h.id = 1
    AND pi.amenity = 'restaurant'
    AND pi.cuisine = 'regional'
    AND st_dwithin(h.localizacion, pi.geom, 1000)
ORDER BY
    distancia_en_metros ASC;
```

	name	opening_hours	distancia_en_metros	geom
1	El Pasaje	[NULL]	251,9371555926	POINT (537
2	Casa Elisa	[NULL]	305,8560740439	POINT (537
3	Fogar de Galicia, Fogar do Santiso	Mo-Su 12:00-1:00	307,3702233518	POINT (537
4	El Papatorio	[NULL]	348,9695505766	POINT (537
5	A Barrola	[NULL]	350,2957705288	POINT (537
6	San Jaime	[NULL]	491,5118787231	POINT (537
7	Artesana Gastrobar	Mo-Sa 13:30-15:30,20:00-23:00	617,8345715444	POINT (536
8	Casa Manolo	[NULL]	754,5258202154	POINT (537



Consulta 8: Un cliente del Hostal de los Reis Católicos quiere ver la 'burbuja' de 500 metros alrededor que muestre en el mapa todos los cajeros (atm) y farmacias (pharmacy) que hay dentro de esa 'burbuja'.

```
select
    distinct
    h.nombre AS nombre_hotel,
    pi.name AS punto_interes,
    pi.amenity AS tipo_servicio,

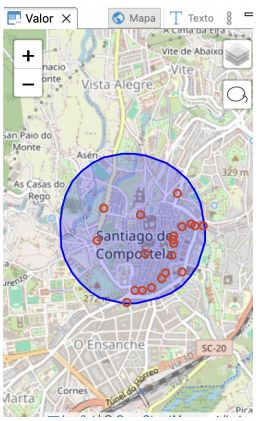
    ST_Buffer(h.localizacion, 500) AS area_hotel, -- La "burbuja" de 500m
    pi.geom AS geometria_punto_interes
FROM
    hoteles h,
    puntos_interes_def pi
WHERE

    h.nombre = 'Hostal dos Reis Católicos'

    AND pi.amenity IN ('atm', 'pharmacy')

    AND ST_DWithin(h.localizacion, pi.geom, 500);
```

	punto_interes	tipo_servicio	area_hotel	geometria_punto_inte
1	Abanca	atm	POLYGON ((536526.1201924	POINT (537293.832932397
2	Abanca	atm	POLYGON ((536526.1201924	POINT (537543.1457180389
3	Farmacia Bescansa	pharmacy	POLYGON ((536526.1201924	POINT (537147.9928208274
4	Farmacia Frade	pharmacy	POLYGON ((536526.1201924	POINT (537451.1061778716
5	Farmacia Gómez-Ulla Astray	pharmacy	POLYGON ((536526.1201924	POINT (537093.2166132594
6	Farmacia Lobelos	pharmacy	POLYGON ((536526.1201924	POINT (536850.0017870368
7	Farmacia Martínez	pharmacy	POLYGON ((536526.1201924	POINT (537373.5706343411
8	Farmacia San Roque	pharmacy	POLYGON ((536526.1201924	POINT (537411.4584364841
9	Farmacia Valdes	pharmacy	POLYGON ((536526.1201924	POINT (537221.6625822699
10	Laura Tatiana de Soto Alberte	pharmacy	POLYGON ((536526.1201924	POINT (537518.9718437232
11	Licenciado Alessandro de Do	pharmacy	POLYGON ((536526.1201924	POINT (537032.1262412745
12	María Inmaculada Ginarte Val	pharmacy	POLYGON ((536526.1201924	POINT (536805.6026483759
13	Santander	atm	POLYGON ((536526.1201924	POINT (537324.5141194271
14	[NULL]	atm	POLYGON ((536526.1201924	POINT (537609.9733359544
15	[NULL]	atm	POLYGON ((536526.1201924	POINT (537132.5911929025
16	[NULL]	atm	POLYGON ((536526.1201924	POINT (537167.7366222595
17	[NULL]	atm	POLYGON ((536526.1201924	POINT (537378.343413658
18	[NULL]	atm	POLYGON ((536526.1201924	POINT (537388.1037120968
19	[NULL]	atm	POLYGON ((536526.1201924	POINT (537447.9566288941
20	[NULL]	pharmacy	POLYGON ((536526.1201924	POINT (537387.4700146638



MODELO RELACIONAL EXTENDIDO

MODELO DE TIPOS COMPUESTOS

Hemos transformado nuestro esquema relacional normalizado en un esquema desnormalizado y jerárquico que se centra en la entidad ***Hotel***. Se basa en una jerarquía de datos anidados creada mediante la definición de tipos de datos compuestos (CREATE TYPE). Los objetos más sencillos pasan a formar parte de estructuras más complejas. ***Direccion*** (calle, ciudad…) pasa a formar parte de ***persona*** que a su vez pasa a formar parte de ***empleado*** y de ***cliente***. De igual forma dentro de ***reserva*** y ***reseña*** se guarda la información del ***cliente***. Además, se guarda la información de la ***habitación*** dentro de la ***reserva***.

Creamos una única tabla ***hotel_agregado*** donde cada fila corresponde a un hotel y en sus columnas se almacenan; además de sus atributos básicos (id, nombre, localización y fecha de inauguración), los arrays de los tipos complejos creados anteriormente: puntos_interes, habitaciones, empleados, reservas, reseñas y clientes. Para cada fila de un hotel en su columna de puntos de interés se almacenan aquellos que están a un radio de un kilómetro.

Finalmente mediante un INSERT extraemos los datos de la base relacional original para cargarlos en la estructura creada. Así toda la información está en una única tabla, optimizando las consultas de lectura al no necesitar realizar varios *joins*.

- **Consulta 1:**

```
--Consulta 1:
select
    ((r.cliente).persona).nombre,
    ((r.cliente).persona).dni,
    (r.habitacion).numero as num_habitacion,
    r.duracion as dias_alojamiento
from
    hotel_agregado h,
    unnest(h.reservas) as r
where
    h.id_hotel = 1
    and (r.habitacion).piso = 5
    and extract(year from r.fecha_entrada) = 2025;
```

	A-Z nombre	A-Z dni	123 num_habitacion	123 dias_alojamiento
1	Vicente Pérez	48783419F	17	14
2	Sergio Fernández	32212400K	105	2
3	Vicente Campos	56608488G	204	2

- **Consulta 2:**

```sql
--Consulta 2:
with ingresos_anuales as (
    select
        extract(year from r.fecha_entrada) as ano,
        sum(r.duracion * (r.habitacion).precio_noche) as ingresos
    from
        hotel_agregado h,
        unnest(h.reservas) as r
    where
        h.id_hotel = 2
    group by
        ano
),
gastos_personal as (
    select
        ia.ano,
        sum(e.sueldo * 12) as total_gastado
    from
        hotel_agregado h,
        unnest(h.empleados) as e,
        ingresos_anuales ia
    where
        h.id_hotel = 2
        and extract(year from e.fecha_incorporacion) <= ia.ano
    group by
        ia.ano
)
select
    i.ano,
    i.ingresos,
    g.total_gastado
from
    ingresos_anuales i
left join
    gastos_personal g on (i.ano = g.ano)
order by
    i.ano desc;
```

	ano	ingresos	total_gastado
1	2.025	62.539	570.312
2	2.024	26.754	570.312
3	2.023	38.355	535.380
4	2.022	18.024	500.100
5	2.021	22.132	449.160
6	2.020	30.546	408.396
7	2.019	34.649	392.196
8	2.018	49.691	380.016
9	2.017	40.234	362.604
10	2.016	29.662	346.692
11	2.015	52.908	309.264

- **Consulta 3:**

```sql
select
    (r.cliente).persona.nombre as nombre_persona, r.valoracion, r.id_resena, r.descripcion
from hoteles.hotel_agregado h,
     UNNEST(h.resenas) as r
where to_tsvector('english', r.descripcion) @@ to_tsquery('english', 'baby')
order by r.valoracion desc;
```

	nombre_persona	valoracion	id_resena	descripcion
1	Pedro Fernández	5	242	great sejour husband sojourned loved, trying hotels paris finally perfect, expect small rooms smaller bathrooms
2	Nicolás Méndez	5	373	superb not asked, stayed mayfair 6 days 1 bedroom suite perfect, traveling 10-month old baby looked internet
3	Clara Díaz	5	28	excellente resort resort excellent, went honeymoon just got, multiple restaurants choose, favorite lobster house
4	David Blanco	5	48	wana, england baby, woop woop, stayed hotel late august early sept family, hotel beautiful staff amazing, met
5	David Santos	4	239	loved place winter wife 1 year old child, loved place room better cuba santa clara resorts room okay, clean nice
6	Vicente Pérez	1	626	not worth price date stay november 16-19 2008my family arrived hotel conference dates listed, arrived decided
7	David Díaz	1	269	horrific despite friendly assistance time booking nothing prepared, booked quad room 2 adults 2 kids baby sta
8	Nicolás Ortega	1	423	not book hotel 5-6 months, arrived decribed building site, picked jaws ground asked builder going directed rec

- **Consulta 4:**

```sql
select distinct
    h.nombre as hotel,
    (c).persona.nombre as cliente,
    (c).dni_cliente
from hoteles.hotel_agregado h,
     UNNEST(h.clientes) as c
where
    (
        select COUNT(*)
        from UNNEST(h.reservas) as r
        where (r.cliente).dni_cliente = (c).dni_cliente
    ) > 5
    or
    (
        select COUNT(*)
        from UNNEST(h.resenas) AS re
        where (re.cliente).dni_cliente = (c).dni_cliente
    ) > 5
order by h.nombre, cliente;
```

	hotel	cliente	dni_cliente
1	Hostal dos Reis Católicos	Alejandra Soto	76184458D
2	Hostal dos Reis Católicos	Iván Vega	94415651E
3	Hostal dos Reis Católicos	Jesús García	68675158D
4	Hotel Puerta del Camino	Alfonso Blanco	13246617X
5	Hotel Puerta del Camino	Cristina Díaz	86706296A
6	Hotel Puerta del Camino	Juan Sánchez	77368169P
7	Hotel Puerta del Camino	Rodrigo Blanco	27482426R

- **Consulta 5:**

```
select *
from (
    select
        h.id_hotel,
        h.nombre as nombre_hotel,
        hab.numero as num_habitacion,
        hab.tipo,
        hab.capacidad,
        (select AVG(r.valoracion)
         from UNNEST(h.resenas) as r
        ) as media_valoracion,
        RANK() over (
            partition by h.id_hotel
            order by
                (select COUNT(*) from UNNEST(h.reservas) as r
                 where (r.habitacion).numero = hab.numero
                   and r.hotel = h.id_hotel
                ) desc,
                (select AVG(r.valoracion) from UNNEST(h.resenas) as r) desc
        ) as ranking
    from hoteles.hotel_agregado h,
         UNNEST(h.habitaciones) as hab  -- desanidamos
) as t
where t.ranking <= 3
order by t.id_hotel, t.ranking;
```

	id_hotel	nombre_hotel	num_habitacion	tipo	capacidad	media_valoracion	ranking
1	1	Hotel Araguaney	143	Doble	2	3,9897435897	1
2	1	Hotel Araguaney	260	Familiar	4	3,9897435897	1
3	1	Hotel Araguaney	17	Individual	1	3,9897435897	3
4	1	Hotel Araguaney	68	Doble	2	3,9897435897	3
5	1	Hotel Araguaney	53	Doble	2	3,9897435897	3
6	1	Hotel Araguaney	47	Doble	2	3,9897435897	3
7	2	Hostal dos Reis Católicos	101	Doble	2	3,936329588	1
8	2	Hostal dos Reis Católicos	269	Familiar	4	3,936329588	1
9	2	Hostal dos Reis Católicos	192	Doble	2	3,936329588	3
10	2	Hostal dos Reis Católicos	237	Suite	2	3,936329588	3
11	2	Hostal dos Reis Católicos	45	Doble	2	3,936329588	3
12	2	Hostal dos Reis Católicos	151	Doble	2	3,936329588	3
13	2	Hostal dos Reis Católicos	308	Doble	2	3,936329588	3
14	3	Hotel Puerta del Camino	152	Doble	2	3,9582089552	1
15	3	Hotel Puerta del Camino	132	Doble	2	3,9582089552	2
16	3	Hotel Puerta del Camino	460	Doble	2	3,9582089552	2

- **Consulta 6:**

	empleado	cargo_empleado	dni_jefe	cargo_jefe
59	72640861M	Botones	56144992N	Jefe de Recepción
60	74081359Y	Auxiliar de Lavandería	92200081V	Jefa de Limpieza
61	75408255E	Técnico de Mantenimiento	06263162D	Jefe de Mantenimiento
62	75782746V	Camarero	07810360J	Jefe de Cocina
63	76661696T	Cocinero	07810360J	Jefe de Cocina
64	77843978N	Auxiliar de Lavandería	92200081V	Jefa de Limpieza
65	77851923X	Botones	77054112P	Jefe de Recepción
66	79680301Q	Camarero	07810360J	Jefe de Cocina
67	80353691E	Camarero	07810360J	Jefe de Cocina
68	80371693R	Auxiliar de Lavandería	91805002B	Jefa de Limpieza
69	81324672T	Botones	77054112P	Jefe de Recepción
70	81869485J	Limpiador/a	45988552N	Jefa de Limpieza
71	81954469W	Técnico de Mantenimiento	33916872W	Jefe de Mantenimiento
72	85036444H	Botones	56144992N	Jefe de Recepción
73	85080767E	Recepcionista	72009195L	Jefe de Recepción
74	86463533N	Botones	77054112P	Jefe de Recepción
75	89015418J	Camarero	07810360J	Jefe de Cocina
76	89395610G	Limpiador/a	45988552N	Jefa de Limpieza
77	90831903V	Técnico de Mantenimiento	06263162D	Jefe de Mantenimiento
78	91636966F	Cocinero	45526763F	Jefe de Cocina
79	92746301N	Limpiador/a	91805002B	Jefa de Limpieza
80	92878800S	Cocinero	07810360J	Jefe de Cocina
81	93653544M	Técnico de Mantenimiento	33916872W	Jefe de Mantenimiento
82	93752471L	Técnico de Mantenimiento	33916872W	Jefe de Mantenimiento
83	95432838M	Técnico de Mantenimiento	42292514H	Jefe de Mantenimiento
84	95759495T	Camarero	45526763F	Jefe de Cocina
85	95796262C	Botones	72009195L	Jefe de Recepción

```
with empleados_sin_subordinados as (
    select h.id_hotel,
           e.dni_empleado,
           e.cargo as cargo_empleado,
           e.dni_director as dni_jefe
    from hoteles.hotel_agregado h,
         UNNEST(h.empleados) as e
    where e.dni_empleado not in (
        select distinct sub.dni_director
        from hoteles.hotel_agregado h2,
             UNNEST(h2.empleados) as sub
        where sub.dni_director is not null
    )
)
select
    e.dni_empleado as empleado,
    e.cargo_empleado,
    e.dni_jefe,
    jefe.cargo as cargo_jefe
from empleados_sin_subordinados e
left join lateral (
    select d.cargo
    from hoteles.hotel_agregado h,
         UNNEST(h.empleados) as d
    where d.dni_empleado = e.dni_jefe
    limit 1
) as jefe on true
order by e.dni_empleado;
```

- **Consulta 7:**

```
--Consulta 7:
select distinct
 pi.name,
 pi.opening_hours,
 ST_Distance(h.localizacion, pi.geom) as distancia_en_metros,
 pi.geom,
 h.localizacion AS geom_hotel
from
 hotel_agregado h,
 unnest(h.puntos_interes) as pi
where
 h.id_hotel = 1
 and pi.amenity = 'restaurant'
 and pi.cuisine = 'regional'
order by
 distancia_en_metros asc;
```

	A-Z name	A-Z opening_hours	123 distancia_en_metros	geom
1	El Pasaje	[NULL]	251,9371555926	POINT (537103.6475774
2	Casa Elisa	[NULL]	305,8560740439	POINT (537123.3523719
3	Fogar de Galicia, Fogar do Santiso	Mo-Su 12:00-1:00	307,3702233518	POINT (537122.5161791
4	El Papatorio	[NULL]	348,9695505766	POINT (537133.034525
5	A Barrola	[NULL]	350,2957705288	POINT (537143.908682
6	San Jaime	[NULL]	491,5118787231	POINT (537179.1246166
7	Artesana Gastrobar	Mo-Sa 13:30-15:30,20:00-23:00	617,8345715444	POINT (536523.224976
8	Casa Manolo	[NULL]	754,5258202154	POINT (537403.390419

- **Consulta 8:**

```
--Consulta 8:
 select distinct
  h.nombre as nombre_hotel,
  pi.name as punto_interes,
  pi.amenity as tipo_servicio,
  ST_Buffer(h.localizacion, 500) as area_hotel,
  pi.geom as geometria_punto_interes
rom
  hotel_agregado h,
  UNNEST(h.puntos_interes) as pi
here
  h.nombre = 'Hostal dos Reis Católicos'
  and pi.amenity in ('atm', 'pharmacy')
  and ST_DWithin(h.localizacion, pi.geom, 500);
```

	A-Z nombre_hotel	A-Z punto_interes	A-Z tipo_servicio	area_hotel
	Hostal dos Reis Católicos	Abanca	atm	POLYGON ((536526.1201924506 4747
2	Hostal dos Reis Católicos	Abanca	atm	POLYGON ((536526.1201924506 4747
3	Hostal dos Reis Católicos	Farmacia Bescansa	pharmacy	POLYGON ((536526.1201924506 4747
4	Hostal dos Reis Católicos	Farmacia Frade	pharmacy	POLYGON ((536526.1201924506 4747
5	Hostal dos Reis Católicos	Farmacia Gómez-Ulla Astray	pharmacy	POLYGON ((536526.1201924506 4747
6	Hostal dos Reis Católicos	Farmacia Lobelos	pharmacy	POLYGON ((536526.1201924506 4747
7	Hostal dos Reis Católicos	Farmacia Martínez	pharmacy	POLYGON ((536526.1201924506 4747

CONSULTA	TIEMPO	OBSERVACIÓN
Consulta 1 - Lámpara rota	0.005s	PERJUDICADA: Se necesita encontrar una reserva determinada pero no se sabe el cliente. Habría que entrar en un hotel y escaner el array de reservas hasta encontrar la fecha dada, para un hotel de muchas reservas es muy lento.
Consulta 2 - Estadística de ingresos y gastos	0.072s	BENEFICIADA: En el modelo relacional esta consulta requiere joins de *hotel*, *reserva* y *empleado,* pero el el agregado se va directamente a la fila del hotel determinado la cual ya tiene cargados un array con las reservas y los empleados. En consecuencia, sumar sueldo y precio_noche únicamente supone acceder a dos listas en memoria RAM.
Consulta 3 - Reseñas sobre bebés	0.339s	PERJUDICADA: La búsqueda de texto dentro de un tipo complejo no es eficiente, tiene que leer y procesar mucho texto, desempaquetar cada reseña para buscar la palabra 'bebé'.
Consulta 4 - Clientes VIP	2.627s	PERJUDICADA: Esta consulta requiere agrupar por dni_cliente en la tabla reservas, pero en el modelo agregado los datos de clientes están dispersos en cada hotel, la base de datos necesita leer cada hotel y desempaquetar todas sus

		reservas, uniéndolas y luego agrupándolas por cliente. Esto es ineficiente.
Consulta 5 - Ranking de habitaciones	2.758s	BENEFICIADA: Ya no se necesita hacer un join de hotel, habitacion, reserva y reseña pues toda la información está en la fila del hotel, solo hay que recorrer el array de reservas.
Consulta 6- Empleados sin subordinado y sus jefes	0.025s	PERJUDICADA: Aunque el tiempo es bajo, estructuralmente es más costosa. Se necesita desanidar a muchos empleados y realizar muchas subconsultas para simular la jerarquía.
Consulta 7 - Restaurantes cercanos a un hotel	0.021 s	BENEFICIADA: Al tener precalculado dentro de cada hotel un array de puntos_interes esta consulta se beneficia mucho porque ya no tiene que hacer los JOIN espaciales, ahora simplemente se busca la fila del hotel 'Araguaney' y se cojen los puntos de interés de esa fila.
Consulta 8 - Puntos de interés cercanos a un hotel	0.030 s	BENEFICIADA: Igual que en la anterior la consulta se vuelve instantánea, ya no se necesitan JOINS sino un select de una columna.

MODELO JSON

El modelo de agregados de JSON de nuestra base de datos va a tomar otra dirección distinta que la del modelo de tipos compuestos. En este caso, en vez de centrarnos en los hoteles, nos centraremos en los **clientes**. Para ello, nuestro modelo JSON constará de una única tabla de Clientes, en la que cada fila contendrá un cliente con sus respectivos datos:

- Los **datos del cliente**: dni, nombre y dirección (vienen de la tabla de Persona).
- El **historial de reservas** tendrá la información de cada reserva hecha por ese cliente en uno de los hoteles (id de reserva, fecha de entrada, duración y número de habitación). En la información acerca del hotel en el que se hizo la reserva, habrá la información completa de cada hotel: el id de hotel, el nombre, las habitaciones que tiene ese hotel, los empleados y los puntos de interés cercanos a él. En las habitaciones estará la información completa de cada una de ellas, así como los empleados y los puntos de interés, que serán objetos JSON dentro del objeto JSON de la información completa del hotel.
- El **historial de reseñas** en el que aparecerán todas las reservas hechas por cada cliente. Cada una de las reservas será un objeto JSON.

CONSULTAS

- **Consulta 1**

```
select
    doc.nombre AS nombre,
    doc.dni_cliente AS dni,
    habitacion ->> 'numero' AS num_habitacion,
    reserva ->> 'duracion' AS dias_alojamiento
from
    modelo_json.documentos_cliente doc, jsonb_array_elements(doc.historial_reservas) AS reserva,
    jsonb_array_elements(reserva -> 'hotel_reservado_info_completa' -> 'habitaciones') AS habitacion
where
    extract(year from TO_DATE(reserva ->> 'fecha_entrada', 'YYYY-MM-DD')) = 2025 and (reserva -> 'hotel_reservado_info_completa' ->> 'id_hotel')::int = 1
    and (habitacion ->> 'piso')::int = 5 and (reserva ->> 'num_habitacion_reservada')::int = (habitacion ->> 'numero')::int;
```

	nombre	dni	num_habitacion	dias_alojamiento
1	Vicente Campos	56608488G	204	2
2	Vicente Pérez	48783419F	17	14
3	Sergio Fernández	32212400K	105	2

- **Consulta 2**

```
with ingresos_anuales as (
    select extract(year from to_date(reserva ->> 'fecha_entrada', 'YYYY-MM-DD')) as ano,
        sum( (reserva ->> 'duracion')::numeric * (hab ->> 'precio_noche')::numeric ) as ingresos
    from
        modelo_json.documentos_cliente doc, jsonb_array_elements(doc.historial_reservas) as reserva,
        jsonb_array_elements(reserva -> 'hotel_reservado_info_completa' -> 'habitaciones') as hab
    where
        (reserva -> 'hotel_reservado_info_completa' ->> 'id_hotel')::int = 2 and (reserva ->> 'num_habitacion_reservada')::int = (hab ->> 'numero')::int
    group by ano
),
gastos_personal as (
    select ia.ano, sum(e.sueldo * 12) as total_gastado
    from
        ingresos_anuales as ia,
        (select distinct on (emp ->> 'dni') (emp ->> 'sueldo')::numeric as sueldo, emp ->> 'fecha_incorporacion' as fecha_incorporacion
            from modelo_json.documentos_cliente doc, jsonb_array_elements(doc.historial_reservas) as reserva,
                jsonb_array_elements(reserva -> 'hotel_reservado_info_completa' -> 'plantilla') as emp
            where (reserva -> 'hotel_reservado_info_completa' ->> 'id_hotel')::int = 2) as e
    where
        extract(year from to_date(e.fecha_incorporacion, 'YYYY-MM-DD')) <= ia.ano
    group by ia.ano)
select i.ano, i.ingresos, g.total_gastado
from ingresos_anuales i
left join gastos_personal g on i.ano = g.ano
order by i.ano desc
limit 15;
```

	123 ano	123 ingresos	123 total_gastado
1	2.025	62.539	570.312
2	2.024	26.754	570.312
3	2.023	38.355	535.380
4	2.022	18.024	500.100
5	2.021	22.132	449.160
6	2.020	30.546	408.396
7	2.019	34.649	392.196
8	2.018	49.691	380.016
9	2.017	40.234	362.604
10	2.016	29.662	346.692
11	2.015	52.908	309.264
12	2.014	33.729	309.264
13	2.013	52.636	225.984
14	2.012	36.874	169.236
15	2.011	31.151	132.024

- **Consulta 3**

```
select doc.nombre as nombre, (reseña ->> 'valoracion')::int as valoracion, reseña ->> 'texto' as descripcion
from modelo_json.documentos_cliente doc, jsonb_array_elements(doc.mis_reseñas) as reseña
where to_tsvector('english', reseña ->> 'texto') @@ to_tsquery('english', 'baby')
order by valoracion desc;
```

	A-Z nombre	123 valoracion	A-Z descripcion
1	David Blanco	5	wana, england baby, woop woop, stayed hotel late august early sept family, hotel beautiful staff amazing
2	Nicolás Méndez	5	superb not asked, stayed mayfair 6 days 1 bedroom suite perfect, traveling 10-month old baby looked int
3	Clara Díaz	5	excellente resort resort excellent, went honeymoon just got, multiple restaurants choose, favorite lobster h
4	Pedro Fernández	5	great sejour husband sojourned loved, trying hotels paris finally perfect, expect small rooms smaller bathr
5	David Santos	4	loved place winter wife 1 year old child, loved place room better cuba santa clara resorts room okay, clea
6	David Díaz	1	horrific despite friendly assistance time booking nothing prepared, booked quad room 2 adults 2 kids bal
7	Nicolás Ortega	1	not book hotel 5-6 months, arrived decribed building site, picked jaws ground asked builder going direct
8	Vicente Pérez	1	not worth price date stay november 16-19 2008my family arrived hotel conference dates listed, arrived de

- **Consulta 4**

```
select reserva -> 'hotel_reservado_info_completa' ->> 'nombre' as hotel, doc.nombre as cliente, doc.dni_cliente
from modelo_json.documentos_cliente doc, jsonb_array_elements(doc.historial_reservas) as reserva
group by hotel, cliente, doc.dni_cliente
having COUNT(*) > 5
union
select reseña ->> 'hotel_nombre' as hotel, doc.nombre as cliente, doc.dni_cliente
from modelo_json.documentos_cliente doc, jsonb_array_elements(doc.mis_reseñas) as reseña
group by hotel, cliente, doc.dni_cliente
having COUNT(*) > 5;
```

	hotel	cliente	dni_cliente
1	Hostal dos Reis Católicos	Alejandra Soto	76184458D
2	Hostal dos Reis Católicos	Iván Vega	94415651E
3	Hostal dos Reis Católicos	Jesús García	68675158D
4	Hotel Puerta del Camino	Alfonso Blanco	13246617X
5	Hotel Puerta del Camino	Cristina Díaz	86706296A
6	Hotel Puerta del Camino	Juan Sánchez	77368169P
7	Hotel Puerta del Camino	Rodrigo Blanco	27482426R

- **Consulta 5**

Esta consulta no se puede solucionar, porque en el modelo relacional accedemos a todas las habitaciones de nuestra base de datos, con o sin reservas; pero con la base de datos de JSON, solo podemos acceder a aquellas habitaciones que estén asociadas a un cliente, por lo que no podremos resolver esta consulta correctamente.

- **Consulta 6**

```
with recursive censo_empregados as (
    select distinct on (emp ->> 'dni') emp ->> 'dni' as dni_empleado, emp ->> 'cargo' as cargo, emp ->> 'dni_director' as dni_director
    from modelo_json.documentos_cliente doc, jsonb_array_elements(doc.historial_reservas) as reserva,
        jsonb_array_elements(reserva -> 'hotel_reservado_info_completa' -> 'plantilla') as emp),
empleados_sin_subordinados as (
    select *
    from censo_empregados e
    where e.dni_empleado not in (
        select distinct dni_director
        from censo_empregados
        where dni_director is not null)),
jerarquia as (
    select e.dni_empleado as empleado, e.cargo as cargo_empleado, e.dni_director as dni_jefe, 1 as nivel
    from empleados_sin_subordinados e
    union all
    select j.empleado, j.cargo_empleado, c.dni_director as dni_jefe, j.nivel + 1
    from jerarquia j join censo_empregados c on j.dni_jefe = c.dni_empleado
    where j.dni_jefe is not null)
select j.empleado, j.cargo_empleado, j.dni_jefe, c.cargo as cargo_jefe
from jerarquia j left join censo_empregados c on j.dni_jefe = c.dni_empleado
where j.nivel = 1
order by j.empleado;
```

	empleado	cargo_empleado	dni_jefe	cargo_jefe
1	00651254V	Recepcionista	56144992N	Jefe de Recepción
2	02772688J	Técnico de Mantenimiento	06263162D	Jefe de Mantenimiento
3	02841500V	Técnico de Mantenimiento	42292514H	Jefe de Mantenimiento
4	02999244L	Botones	72009195L	Jefe de Recepción
5	04391822Y	Técnico de Mantenimiento	42292514H	Jefe de Mantenimiento
6	06011941A	Recepcionista	77054112P	Jefe de Recepción
7	06382995S	Recepcionista	77054112P	Jefe de Recepción
8	10527053A	Limpiador/a	45988552N	Jefa de Limpieza
9	10904380S	Auxiliar de Lavandería	45988552N	Jefa de Limpieza
10	11939686X	Recepcionista	72009195L	Jefe de Recepción
11	12670662Z	Técnico de Mantenimiento	42292514H	Jefe de Mantenimiento
12	14353053N	Botones	56144992N	Jefe de Recepción
13	17107946G	Técnico de Mantenimiento	33916872W	Jefe de Mantenimiento
14	17577868Y	Cocinero	02989767E	Jefe de Cocina

- ## Consulta 7

```
select distinct punto_interese ->> 'nome' as name, punto_interese ->> 'horario' as opening_hours,
    (punto_interese ->> 'distancia_aprox_metros')::numeric as distancia_en_metros, ST_GeomFromGeoJSON(punto_interese -> 'localizacion_punto') as geom
from modelo_json.documentos_cliente doc, jsonb_array_elements(doc.historial_reservas) as reserva,
    jsonb_array_elements(reserva -> 'hotel_reservado_info_completa' -> 'puntos_interes_cercanos') as punto_interese
where (reserva -> 'hotel_reservado_info_completa' ->> 'id_hotel')::int = 1
    and punto_interese ->> 'tipo_servizo' = 'restaurant' and punto_interese ->> 'cociña' = 'regional'
order by distancia_en_metros asc;
```

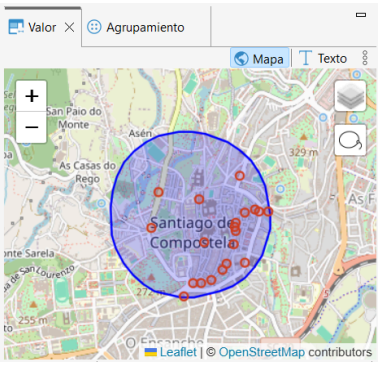
	name	opening_hours	distancia_en_metros	geom
1	El Pasaje	[NULL]	251,94	POINT (537103.647577438 4747317.826960548)
2	Casa Elisa	[NULL]	305,86	POINT (537123.352371966 4747368.272007059)
3	Fogar de Galicia, Fogar do Santiso	Mo-Su 12:00-1:00	307,37	POINT (537122.516179173 4747370.366327159)
4	El Papatorio	[NULL]	348,97	POINT (537133.034525419 4747411.389251107)
5	A Barrola	[NULL]	350,3	POINT (537143.908682249 4747407.683402764)
6	San Jaime	[NULL]	491,51	POINT (537179.124616654 4747546.807682647)
7	Artesana Gastrobar	Mo-Sa 13:30-15:30,20:00-23:00	617,83	POINT (536523.224976199 4746601.184032382)
8	Casa Manolo	[NULL]	754,53	POINT (537403.390419909 4747723.926347004)



- ## Consulta 8

```
select distinct (punto_interese ->> 'nome') as punto_interes, (punto_interese ->> 'tipo_servizo') as tipo_servicio,
    ST_Buffer(ST_GeomFromGeoJSON(reserva -> 'hotel_reservado_info_completa' -> 'localizacion_hotel'), 500) as area_hotel,
    ST_GeomFromGeoJSON(punto_interese -> 'localizacion_punto') as geometria_punto_interes
from modelo_json.documentos_cliente doc, jsonb_array_elements(doc.historial_reservas) as reserva,
    jsonb_array_elements(reserva -> 'hotel_reservado_info_completa' -> 'puntos_interes_cercanos') as punto_interese
where (reserva -> 'hotel_reservado_info_completa' ->> 'nombre') = 'Hostal dos Reis Católicos' and (punto_interese ->> 'tipo_servizo') in ('atm', 'pharmacy')
    and (punto_interese ->> 'distancia_aprox_metros')::numeric <= 500;
```

	punto_interes	tipo_servicio	area_hotel	geometria_punto_interes
1	Abanca	atm	POLYGON ((536526.1201924506 4747805.807480	POINT (537293.832932397 474
2	Abanca	atm	POLYGON ((536526.1201924506 4747805.807480	POINT (537543.145718039 474
3	Farmacia Bescansa	pharmacy	POLYGON ((536526.1201924506 4747805.807480	POINT (537147.992820827 474
4	Farmacia Frade	pharmacy	POLYGON ((536526.1201924506 4747805.807480	POINT (537451.106177872 474
5	Farmacia Gómez-Ulla Astray	pharmacy	POLYGON ((536526.1201924506 4747805.807480	POINT (537093.216613259 474
6	Farmacia Lobelos	pharmacy	POLYGON ((536526.1201924506 4747805.807480	POINT (536850.001787037 474
7	Farmacia Martínez	pharmacy	POLYGON ((536526.1201924506 4747805.807480	POINT (537373.570634341 474
8	Farmacia San Roque	pharmacy	POLYGON ((536526.1201924506 4747805.807480	POINT (537411.458436484 474
9	Farmacia Valdes	pharmacy	POLYGON ((536526.1201924506 4747805.807480	POINT (537221.66258227 4747
10	Laura Tatiana de Soto Alberte	pharmacy	POLYGON ((536526.1201924506 4747805.807480	POINT (537518.971843723 474
11	Licenciado Alessandro de Donno	pharmacy	POLYGON ((536526.1201924506 4747805.807480	POINT (537032.126241274 474
12	María Inmaculada Ginarte Val	pharmacy	POLYGON ((536526.1201924506 4747805.807480	POINT (536805.602648376 474
13	Santander	atm	POLYGON ((536526.1201924506 4747805.807480	POINT (537324.514119427 474
14	[NULL]	atm	POLYGON ((536526.1201924506 4747805.807480	POINT (537609.973335954 474
15	[NULL]	atm	POLYGON ((536526.1201924506 4747805.807480	POINT (537132.591192902 474
16	[NULL]	atm	POLYGON ((536526.1201924506 4747805.807480	POINT (537167.73662226 4747



- ## Rendimiento de consultas

CONSULTA	TIEMPO	RENDIMIENTO
Consulta 1	8.922s	Esta consulta es **muy ineficiente** porque tenemos que, para cada cliente, expandir sus reservas; para cada reserva, expandir las habitaciones del hotel de la reserva. Por ello **generamos muchas filas temporales por cliente**, lo cual es muy costoso computacionalmente.
Consulta 2	19s	La consulta es **muy ineficiente** porque repetimos una gran cantidad de información repetida. Para calcular los **gastos** del hotel especificado, tenemos que buscar todos los clientes alojados en ese hotel. De cada una de las reservas de ese cliente hay que sacar la lista completa de empleados. Después realiza un **distinct on**, lo que tira muchos datos seleccionados a la basura. Para los **ingresos** se toma la reserva, se extraen todas las habitaciones del hotel para encontrar la habitación seleccionada y coger el precio.
Consulta 3	0.090s	Esta consulta es **eficiente** con el esquema JSON porque generamos muchas menos filas temporales que con las consultas anteriores. Solo tenemos que acceder al cliente, a su reserva y a la reseña de la misma.
Consulta 4	2.884s	Esta consulta es **moderadamente eficiente** porque sólo accedemos a las reservas de cada cliente. Lo que resulta algo costoso es que con la cláusula del **having** la base de datos debe

		coger todos los clientes de la tabla, descomprimir su JSON, coger todas sus reservas, agruparlas, contarlas y después descartar los que tienen menos de 5, por lo que **creamos muchos datos temporales innecesarios**.
Consulta 6	5.414s	Esta consulta es **muy ineficiente** porque para acceder a los empleados hay que seleccionar de cada cliente todas las reservas y de cada reserva desplegar la plantilla de empleados del hotel. Además, con el **distinct on** eliminamos muchos datos que eran innecesarios. La consulta recursiva se desaprovecha porque se crea todo el árbol de niveles, pero solo se selecciona el primer nivel, descartando el resto de los datos.
Consulta 7	9.954s	Esta consulta también es **ineficiente** porque para acceder a los puntos de interés, tenemos que acceder a cada reserva de cada cliente. A mayores, si tenemos muchas reservas de un hotel, los puntos de interés son los mismos para este hotel, por lo que usamos datos innecesariamente.
Consulta 8	29s	Esta consulta es, de nuevo, **muy ineficiente** porque tenemos que acceder a todas las reservas asociadas al hostal, en las cuales hay la misma información sobre los puntos de interés, por lo que **habrá muchas copias innecesarias de esa información que después se borrarán.**

- **Comparación entre relacional/agregados/JSON + resumen final de rendimiento (cuáles van mejor con un modelo y cuáles con otro). Subrayamos en verde el mejor modelo.**

	RELACIONAL	**AGREGADOS**	**JSON**
Número de tablas	Una tabla por cada entidad	Única tabla: Hoteles	Única tabla: Clientes
Consulta 1	**Eficiente al tener los joins bien definidos**	Perjudicada al tener que escanear todo el array de reservas	Muy ineficiente al tener que expandir la reservas de cada cliente
Consulta 2	Eficiente pero necesita muchos joins	**Muy beneficiada, ya que toda la información está en la fila del hotel**	Muy lenta
Consulta 3	Normal, una eficiencia estándar	Perjudicada al tener que desempaquetar las reseñas	**Beneficiada gracias a la estructura por cliente**
Consulta 4	**Normal, agregación estándar**	Perjudicada, clientes dispersos por hotel, requiere escanear todos los hoteles	**Normal, hay que expandir el JSON pero solo por cliente**
Consulta 5	Eficiente pero necesita muchos joins y agregaciones	**Muy beneficiada ya que todo está en la fila de hotel**	————————————
Consulta 6	**Normal, agregación estándar**	Muy perjudicada ya que hay que desempaquetar los empleados que se encuentran dentro de Hotel	Muy ineficiente, se desaprovecha la consulta recursiva
Consulta 7	Normal, agregación estándar	**Muy beneficiada ya que tenemos los puntos de interés dentro de cada hotel precalculados y no se necesitan JOIN espaciales.**	Muy lenta e ineficiente al tener que acceder a cada reserva de cada cliente
Consulta 8	Normal, agregación estándar	**Muy beneficiada, como la anterior, no necesitan JOIN y se vuelve instantánea**	Muy lenta e ineficiente al tener muchas copias innecesarias de los puntos de interés por cliente.

BASE DE DATOS RELACIONAL DISTRIBUÍDA

Para crear la base de datos distribuída primero creamos el clúster con un nodo coordinador y dos workers, después damos de alto los nodos en el coordinador, creamos el esquema relacional y por último se distribuyen o replican las tablas.

A la hora de decidir qué tablas distribuír y cuales replicar hay que tener en cuenta los *joins* de las consultas, los datos que se unen frecuentemente deberían estar en el mismo *worker*, evitando que se muevan datos por la red, que es la operación más costosa.

La mayoría de nuestras consultas giran en torno a un hotel en concreto (el *join* suele hacerse por *id_hotel*) , por ello para distribuír los datos de forma eficiente hemos dividido las tablas (**hoteles, habitaciones, empleados, reservas, reseñas**) entre los workers en función de su *id_hotel*. De esta forma la mayoría de joins se ejecutan dentro del mismo *worker*.

Las tablas ***personas***, ***clientes*** se unen usando dni y no id del hotel, al replicarlas en todas las tablas esos joins pueden hacerse dentro de cada nodo, lo mismo ocurre con la tabla de ***puntos_interes***, la cual al replicarse en todos los nodos evita que haya que mover datos geográficos por la red.

Tablas de referencia	personas, clientes, puntos_interes
Tablas distribuidas	hoteles, habitaciones, empleados, reservas, resenas

La tabla ***empleado*** presenta un conflicto, no podemos hacer la tabla de referencia porque tiene una clave foránea a hoteles que es distribuída y Citus solo permite que la tablas de referencia tengan claves foráneas a otras tablas de referencia. Pero tampoco puede ser distribuída porque su clave primaria (*dni_empleado*) y foránea a sí misma no son compatibles con la distribución. La solución ha sido modificar el esquema de la tabla para que puede distribuírse eliminado la relación reflexiva, por ello la consulta 3 no se puede ejecutar en esta base de datos distribuída.

1. Análisis del impacto de parar un worker

Los datos de las tablas distribuídas (***hoteles, habitaciones, empleados, reservas, reseñas***) están repartidos en los *workers*. Al parar *hoteles-worker1* cualquier consulta que necesitase datos que estaban en ese *worker* no va a funcionar porque no se puede acceder a esos datos. Las consultas que solo usen las tablas replicadas o si los datos que necesitan están en el *worker2* seguirán funcionando.

Las consultas que utilizan agregación o necesitan acceder a todos los datos dan error porque Citus detecta que faltan shards, por ejemplo la consulta 4 (clientes VIP)

necesita leer las reservas de todos los hoteles, la consulta 5 (ranking habitaciones) necesita acceder a datos de habitaciones de los tres hoteles.

La estrategia de distribución Citus permite escalabilidad y rendimiento pero no tiene tolerancia a fallos, si falla un *worker*, los shards o particiones quedan inaccesibles y las consultas que los requieran no van a funcionar.

2. Formato columnar

El objetivo del formato columnar es optimizar el rendimiento a la hora de hacer consultas analíticas como agregaciones sobre una gran cantidad de datos pues permite acceder a columnas específicas de una tabla sin tener que leer la fila completa. En nuestro proyecto las tablas que suponen ventajas si se almacenan en formato columnar son las ***reservas*** y las ***reseñas***, ambas tablas acumulan un gran volumen de datos que en un caso real iría en aumento con el tiempo.

CONSULTA	TIEMPO DISTRIBUIDA	TIEMPO CON FORMATO COLUMNAR	OBSERVACIÓN
Consulta 1	0,010s	0,054s	PERJUDICADA: es una consulta transaccional que busca filas específicas, el formato columnar no es bueno para este tipo de consultas porque para devolver la fila completa de la tabla ***reserva*** tiene que ir cogiendo un trozo de cada columna almacenadas por separado y unirlos, lo que lo hace más lento.
Consulta 2	0,010s	0.09s	BENEFICIADA: utiliza operaciones de agregación, concretamente un sum() sobre las columnas *precio_noche* y *duracion* de la tabla ***reserva***. Esto se beneficia del formato columnar porque solo accede a esas columnas ignorando el resto de datos.
Consulta 3	0,132s	0,118s	BENEFICIADA: Buscar en *descripcion* la palabra "*bebé"* solo necesita leer una columna de la tabla, el formato columnar hace que solo se lea esa columna de disco ignorando las demás columnas y haciendo la consulta más rápida.
Consulta 4	0,040s	0,039s	BENEFICIADA: utiliza una operación de agregación, count(), el formato columnar permite llevarlo a cabo con mucha más rapidez.
Consulta 5	3s	0,7s	BENEFICIADA: utiliza avg(valoracion) en ***reseñas*** y count(num_reserva) en ***reserva***, operaciones de agregación que se benefician del formato columnar.
Consulta 7	0,010s	0,010s	No le afecta porque no usa las tablas ***reseñas*** y ***reservas***.
Consulta 8	0,050s	0,050s	No le afecta porque no usa las tablas ***reseñas*** y ***reservas***.

BASE DE DATOS NoSQL DOCUMENTAL: MONGODB

Para la creación de la base de datos documental con MongoDB vamos a usar nuestro esquema JSON creado anteriormente, con la misma dirección de agregación, obteniendo una super tabla de clientes que contenga todos los datos agrupados. Para empezar, tenemos que exportar el esquema json de nuestra base de datos agregada con JSON. Para ello hay que entrar en el contenedor en el que creamos la base de datos de JSON (que en este caso es el contenedor “edge-gria-pgsql”). Una vez copiado el esquema de la base de datos, entramos en el contenedor del broker de MongoDB e importamos los datos del esquema exportado previamente.

Ahora vamos a crear el cluster de MongoDB. Para ello creamos **tres máquinas principales** para seguir el esquema de cluster establecido y entramos en ellas.

Ahora vamos a crear el **Config Server** en las 3 máquinas. El Config Server se encarga de decirle al clúster dónde están los datos y cómo están organizados. Por lo tanto, creamos el proceso que le corresponde con un nuevo puerto (en este caso el 27017) y activamos el proceso para todas las máquinas.

Creamos ahora el proceso de **mongos**, que se encargará de hacer más sencillas las consultas que queramos hacer sobre el cluster, dejando de lado la complejidad de todos sus nodos. Este proceso irá en el siguiente puerto disponible, que es el 27018.

Por último, tenemos que añadir los tres **shards** en los que se particionarán los datos, por lo que primero tenemos que iniciar un proceso por cada shard igual que hemos hecho hasta ahora (como si fuera un replicaset), pero una vez creados los procesos los añadiremos al cluster, y podremos ver como está en las tres máquinas preparados para dividir los datos. Para cada shard usamos los puertos 27019, 27020 y 27021.

```
shards
[
  {
    _id: 'rsSH1',
    host: 'rsSH1/maquina1:27022,maquina2:27022,maquina3:27022',
    state: 1,
    topologyTime: Timestamp({ t: 1763823421, i: 8 }),
    replSetConfigVersion: Long('-1')
  },
```

```
  {
    _id: 'rsSH2',
    host: 'rsSH2/maquina1:27023,maquina2:27023,maquina3:27023',
    state: 1,
    topologyTime: Timestamp({ t: 1763823426, i: 10 }),
    replSetConfigVersion: Long('-1')
  },
```

```
  {
    _id: 'rsSH3',
    host: 'rsSH3/maquina1:27024,maquina2:27024,maquina3:27024',
    state: 1,
    topologyTime: Timestamp({ t: 1763823429, i: 33 }),
    replSetConfigVersion: Long('-1')
  }
]
```

Para comprobar que nuestro cluster está creado correctamente siguiendo el esquema establecido, miramos los procesos iniciados en nuestras máquinas.

```
    PID TTY      STAT   TIME COMMAND
      1 pts/0    Ss+    0:00 bash
     19 ?        Sl     7:14 mongod --configsvr --replSet r
    254 ?        Sl     0:48 mongos --port 27018 --configdb
    323 ?        Sl     5:53 mongod --shardsvr --replSet rs
    399 ?        Sl     5:41 mongod --shardsvr --replSet rs
    474 ?        Sl     6:12 mongod --shardsvr --replSet rs
   4419 pts/1    Ss     0:00 bash
   4426 pts/1    R+     0:00 ps -xa
```

Podemos ver que hay 5 procesos iniciados, que se corresponden con el Config Server, el Mongos y los 3 shards. A continuación comprobamos que el mongos reconoce las partes del cluster (los shards); para ello iniciamos uno de los procesos mongo en una de las máquinas y ejecutamos un **sh.status()**.



Se puede observar que los 3 shards están replicados en las 3 máquinas (como aparece en el apartado “host”), por lo que disponemos de alta disponibilidad. Además, podemos comprobar que actualmente en nuestro cluster hay un proceso mongos en cada máquina.

Ahora que ya tenemos el cluster al completo y preparado para insertar los datos de nuestra base de datos JSON. Para ello debemos exportar el esquema de JSON e importar los datos en nuestro cluster MongoDB. Una vez obtenido el esquema, importamos los datos y comprobamos que estén cargados correctamente.



Podemos ver que hay tantos documentos como clientes, que es la super tabla que creamos en nuestro modelo JSON.

Hay que decidir cómo se particionarán los datos entre los tres shards. Primero tenemos que activar el particionamiento de los datos (**sh.enableSharding("hoteles_db")**). Escogemos el índice por el que particionaremos los datos, que en este caso será el de dni_cliente porque todos los dnis son valores únicos que no se repiten, permitiendo mayor precisión en la partición de los shards. Además, si particionamos por dni_cliente será mucho más fácil y rápido para las consultas acceder al shard correspondiente a una persona con ese dni. Entonces, para ello tenemos que crear el “mapa” en el que se dividirán los datos con el comando **db.clientes.createIndex({ dni_cliente: "hashed" })** (“hashed” es para tener una distribución uniforme y aleatoria en los shards) y después empezar a distribuir los datos en los tres shards con **sh.shardCollection("hoteles_db.clientes", { dni_cliente: "hashed" })** hasta que todos estén equilibrados.

```
Totals
{
  data: '1.54GiB',
  docs: 773,
  chunks: 8,
  'Shard rsSH1': [
    '29 % data',
    '20.18 % docs in cluster',
    '2.95MiB avg obj size on shard'
  ],
  'Shard rsSH3': [
    '25.3 % data',
    '15.13 % docs in cluster',
    '3.43MiB avg obj size on shard'
  ],
  'Shard rsSH2': [
    '45.69 % data',
    '64.68 % docs in cluster',
    '3.19MiB avg obj size on shard'
  ]
}
```

Finalmente, podemos ver que los datos están particionados en los tres shards de una forma más o menos uniforme, gracias al hashing.

Para realizar las consultas debemos conectarnos al proceso de mongos (puerto 27021).

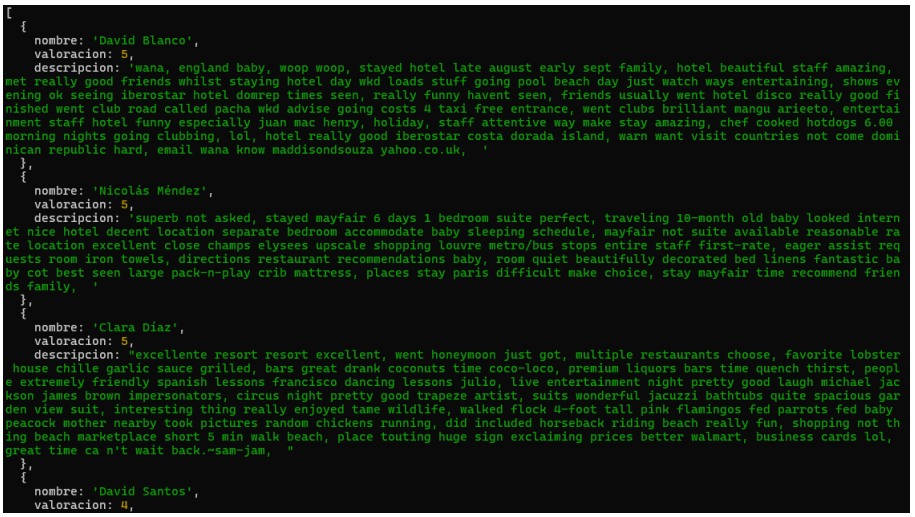
- **Consulta 1**

```
[
  {
    nombre: 'Vicente Pérez',
    dni: '48783419F',
    num_habitacion: 17,
    dias_alojamiento: 14
  },
  {
    nombre: 'Sergio Fernández',
    dni: '32212400K',
    num_habitacion: 105,
    dias_alojamiento: 2
  },
  {
    nombre: 'Vicente Campos',
    dni: '56608488G',
    num_habitacion: 204,
    dias_alojamiento: 2
  }
]
```

- **Consulta 2**

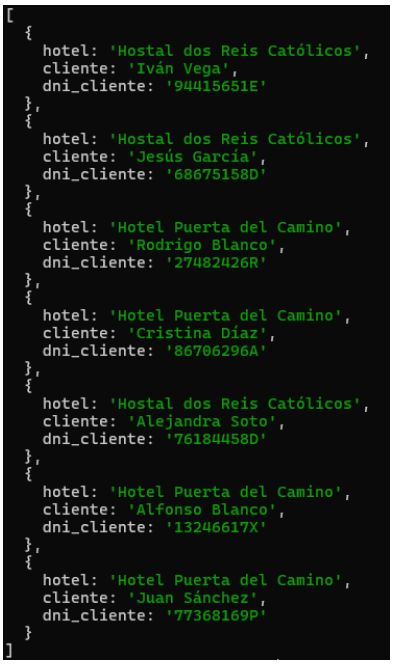
```
[
  { ano: 2025, ingresos: 62539, total_gastado: 570312 },
  { ano: 2024, ingresos: 26754, total_gastado: 570312 },
  { ano: 2023, ingresos: 38355, total_gastado: 535380 },
  { ano: 2022, ingresos: 18024, total_gastado: 500100 },
  { ano: 2021, ingresos: 22132, total_gastado: 449160 },
  { ano: 2020, ingresos: 30546, total_gastado: 408396 },
  { ano: 2019, ingresos: 34649, total_gastado: 392196 },
  { ano: 2018, ingresos: 49691, total_gastado: 380016 },
  { ano: 2017, ingresos: 40234, total_gastado: 362604 },
  { ano: 2016, ingresos: 29662, total_gastado: 346692 },
  { ano: 2015, ingresos: 52908, total_gastado: 309264 },
  { ano: 2014, ingresos: 33729, total_gastado: 309264 },
  { ano: 2013, ingresos: 52636, total_gastado: 225984 },
  { ano: 2012, ingresos: 36874, total_gastado: 169236 },
  { ano: 2011, ingresos: 31151, total_gastado: 132024 }
]
```

- **Consulta 3**



```
[
  {
    nombre: 'David Blanco',
    valoracion: 5,
    descripcion: 'wana, england baby, woop woop, stayed hotel late august early sept family, hotel beautiful staff amazing,
met really good friends whilst staying hotel day wkd loads stuff going pool beach day just watch ways entertaining, shows ev
ening ok seeing iberostar hotel domrep times seen, really funny havent seen, friends usually went hotel disco really good fi
nished went club road called pacha wkd advise going costs 4 taxi free entrance, went clubs brilliant mangu arieeto, entertai
nment staff hotel funny especially juan mac henry, holiday, staff attentive way make stay amazing, chef cooked hotdogs 6.00
morning nights going clubbing, lol, hotel really good iberostar costa dorada island, warn want visit countries not come domi
nican republic hard, email wana know maddisondsouza yahoo.co.uk,  '
  },
  {
    nombre: 'Nicolás Méndez',
    valoracion: 5,
    descripcion: 'superb not asked, stayed mayfair 6 days 1 bedroom suite perfect, traveling 10-month old baby looked intern
et nice hotel decent location separate bedroom accommodate baby sleeping schedule, mayfair not suite available reasonable ra
te location excellent close champs elysees upscale shopping louvre metro/bus stops entire staff first-rate, eager assist req
uests room iron towels, directions restaurant recommendations baby, room quiet beautifully decorated bed linens fantastic ba
by cot best seen large pack-n-play crib mattress, places stay paris difficult make choice, stay mayfair time recommend frien
ds family,  '
  },
  {
    nombre: 'Clara Díaz',
    valoracion: 5,
    descripcion: "excellente resort resort excellent, went honeymoon just got, multiple restaurants choose, favorite lobster
 house chille garlic sauce grilled, bars great drank coconuts time coco-loco, premium liquors bars time quench thirst, peopl
e extremely friendly spanish lessons francisco dancing lessons julio, live entertainment night pretty good laugh michael jac
kson james brown impersonators, circus night pretty good trapeze artist, suits wonderful jacuzzi bathtubs quite spacious gar
den view suit, interesting thing really enjoyed tame wildlife, walked flock 4-foot tall pink flamingos fed parrots fed baby
peacock mother nearby took pictures random chickens running, did included horseback riding beach really fun, shopping not th
ing beach marketplace short 5 min walk beach, place touting huge sign exclaiming prices better walmart, business cards lol,
great time ca n't wait back.~sam-jam,  "
  },
  {
    nombre: 'David Santos',
    valoracion: 4,
```


- **Consulta 4**



```
[
  {
    hotel: 'Hostal dos Reis Católicos',
    cliente: 'Iván Vega',
    dni_cliente: '94415651E'
  },
  {
    hotel: 'Hostal dos Reis Católicos',
    cliente: 'Jesús Garcia',
    dni_cliente: '68675158D'
  },
  {
    hotel: 'Hotel Puerta del Camino',
    cliente: 'Rodrigo Blanco',
    dni_cliente: '27482426R'
  },
  {
    hotel: 'Hotel Puerta del Camino',
    cliente: 'Cristina Díaz',
    dni_cliente: '86706296A'
  },
  {
    hotel: 'Hostal dos Reis Católicos',
    cliente: 'Alejandra Soto',
    dni_cliente: '76184458D'
  },
  {
    hotel: 'Hotel Puerta del Camino',
    cliente: 'Alfonso Blanco',
    dni_cliente: '13246617X'
  },
  {
    hotel: 'Hotel Puerta del Camino',
    cliente: 'Juan Sánchez',
    dni_cliente: '77368169P'
  }
]
```


- **Consulta 5**

Esta consulta no se puede resolver, porque el esquema que usamos para el cluster es el mismo esquema de nuestro modelo de JSON, y con este esquema no podíamos resolver esta consulta anteriormente, ya que no podemos acceder a las habitaciones que no tengan un cliente asociado.

- **Consulta 6**


```
[
  {
    empleado: '00651254V',
    cargo_empleado: 'Recepcionista',
    dni_jefe: '56144992N',
    cargo_jefe: 'Jefe de Recepción'
  },
  {
    empleado: '02772688J',
    cargo_empleado: 'Técnico de Mantenimiento',
    dni_jefe: '06263162D',
    cargo_jefe: 'Jefe de Mantenimiento'
  },
  {
    empleado: '02841500V',
    cargo_empleado: 'Técnico de Mantenimiento',
    dni_jefe: '42292514H',
    cargo_jefe: 'Jefe de Mantenimiento'
  },
  {
    empleado: '02999244L',
    cargo_empleado: 'Botones',
    dni_jefe: '72089195L',
    cargo_jefe: 'Jefe de Recepción'
  },
  {
    empleado: '04391822Y',
    cargo_empleado: 'Técnico de Mantenimiento',
    dni_jefe: '42292514H',
    cargo_jefe: 'Jefe de Mantenimiento'
  },
```


- **Consulta 7**


```
[
  {
    name: 'El Pasaje',
    opening_hours: null,
    distancia_en_metros: 251.94,
    geom: {
      crs: { type: 'name', properties: { name: 'EPSG:25829' } },
      type: 'Point',
      coordinates: [ 537103.647577438, 4747317.826960548 ]
    }
  },
  {
    name: 'Casa Elisa',
    opening_hours: null,
    distancia_en_metros: 305.86,
    geom: {
      crs: { type: 'name', properties: { name: 'EPSG:25829' } },
      type: 'Point',
      coordinates: [ 537123.352371966, 4747368.272007059 ]
    }
  },
  {
    name: 'Fogar de Galicia, Fogar do Santiso',
    opening_hours: 'Mo-Su 12:00-1:00',
    distancia_en_metros: 307.37,
    geom: {
      crs: { type: 'name', properties: { name: 'EPSG:25829' } },
      type: 'Point',
      coordinates: [ 537122.516179173, 4747370.366327159 ]
    }
  },
```


- **Consulta 8**


```
    geometria_punto_interes: {
      type: 'Point',
      coordinates: [ 537378.343413658, 4747672.751779495 ]
    }
  },
  {
    punto_interes: 'Licenciado Alessandro de Donno',
    tipo_servicio: 'pharmacy',
    area_hotel: {
      type: 'Point',
      coordinates: [
        [
          [ 537112.753189724, 4747799.682025311 ],
          [ 537119.654414666, 4747735.673342746 ],
          [ 537122.982851851, 4747705.713105541 ],
          [ 537093.513207748, 4747702.103214461 ],
          [ 537093.34321327, 4747703.623196998 ],
          [ 537092.923218345, 4747703.573198533 ],
          [ 537089.903254986, 4747703.243209226 ],
          [ 537089.573259995, 4747703.693204687 ],
          [ 537087.171541737, 4747703.430008013 ],
          [ 537086.653295344, 4747703.373215037 ],
          [ 537086.423297037, 4747702.863221544 ],
          [ 537083.999266276, 4747702.591834163 ],
          [ 537082.493344591, 4747702.423238585 ],
          [ 537082.66333907, 4747700.903253046 ],
          [ 537052.283706749, 4747697.5333612 ],
          [ 537052.423702274, 4747696.31337521 ],
          [ 537053.143679652, 4747690.213445208 ],
          [ 537036.753877268, 4747688.063507453 ],
          [ 537033.373937406, 4747696.233419153 ],
          [ 537033.603934745, 4747696.313417694 ],
          [ 537033.603934745, 4747696.313417697 ],
          [ 537033.603934745, 4747696.313417697 ],
```

- **Rendimiento de consultas**

Como el esquema usado para el clúster de MongoDB es el mismo que el de nuestra base de datos JSON, entonces, el rendimiento de las consultas será el mismo que con JSON, por lo que no hace falta analizarlas de nuevo.

- **¿Qué ocurre si paramos una de las máquinas?**

Paramos la máquina 1, por ejemplo (**docker stop maquina1**) y entramos en el proceso de mongos para hacer una inserción y una lectura para comprobar que funciona el cluster con una máquina parada.

```
[direct: mongos] hoteles_db> db.clientes.insertOne({ nombre: "Test", fecha: new Date() })
{
  acknowledged: true,
  insertedId: ObjectId('692c22d43f32bfba859dc29d')
}
```

```
[direct: mongos] hoteles_db> db.clientes.findOne()
{
  _id: ObjectId('692b6c45cfd6424620b227f2'),
  nombre: 'Adriana Muñoz',
  dni_cliente: '50911044B',
  'mis_reseñas': [
    {
      texto: 'enjoyable experience stayed christmas period extremely busy friendly informat
ive, think just opened, stayed executive floors slightly disappointed rooms slightly smalle
r usual hotel class, clean tha view upper floors amazing, just hong kong island.the swimmin
g pool floor best views gym level, times hotel breakfast everyday, menu changed day day foo
d good, shopping centre incorporated hotel not far metro station.if want close hand think b
etter hotels kowloon,  ',
      hotel_id: 3,
      valoracion: 4,
      hotel_nombre: 'Hotel Puerta del Camino'
    }
```

Comprobamos que tanto la inserción como la lectura de datos funcionan correctamente.

- **¿Qué ocurre si tenemos paradas dos máquinas?**

Ahora, con la máquina 1 aún parada, paramos la máquina2 (docker stop maquina2) y entramos en el proceso mongos de la máquina 3 para probar a insertar y leer datos.

```
[direct: mongos] hoteles_db> db.clientes.insertOne({ nombre: "Test2" })
MongoServerError: Write results unavailable from failing to target a host in the shard rsSH
3 :: caused by :: Could not find host matching read preference { mode: "primary" } for set
rsSH3
```

```
[direct: mongos] hoteles_db> db.clientes.findOne()
MongoServerError[FailedToSatisfyReadPreference]: Could not find host matching read preferen
ce { mode: "primary" } for set rsSH2
```

Se puede ver que tanto para la escritura como para la lectura el cluster ha dado fallos. Esto se debe a que para aceptar escrituras en el cluster, las tres máquinas deben hacer votación para escoger al primary. Pero al haber dos máquinas paradas, no se puede

llegar a la mayoría de votos, por lo que la máquina 3 al no recibir los votos necesarios para ser primary se convierte en secondary automáticamente. Por lo tanto, no hay primary y no se puede escribir.

En cuanto a la lectura, el cluster no permite las lecturas por la configuración que hay por defecto para ellas. MongoDB tiene una configuración por defecto en la que solo permite leer datos si proceden del primary, ya que son los únicos que considera fiables. Pero en nuestro caso, al no haber primary, da un fallo.

- **Cambios en preferencias y compromisos de lectura**

Como con la configuración por defecto, el cluster no permite las lecturas con dos máquinas paradas, vamos a probar con varios tipos de preferencias y compromisos de lectura.

Modo "primaryPreferred": este modo de lectura usará la máquina establecida como primary, pero si no está disponible usa cualquiera de las restantes, priorizando la disponibilidad. (**db.getMongo().setReadPref("primaryPreferred")**).



Modo "nearest" (lectura por cercanía): este modo de lectura está pensado para clusters distribuidos geográficamente, ya que el cluster coge a la máquina que conteste más rápido. En nuestro caso, como solo tenemos una máquina activa, esta será la más cercana.



Modo “linearizable”: este modo exige que el dato sea el más actual posible y que esté confirmado por la mayoría en ese instante. En nuestro cluster este modo fallará, porque al haber solamente una máquina activa, no podemos obtener la mayoría por votación, por lo que la consulta se bloquea o falla para no proporcionar datos erróneos.

```
[direct: mongos] hoteles_db> db.getMongo().setReadConcern("linearizable")

[direct: mongos] hoteles_db> db.clientes.findOne()
MongoServerError[HostUnreachable]: cannot satisfy linearizable read concern on non-primary
node
```

NEO4J

En este apartado diseñaremos el modelo de grafos para Neo4J.
Para esto necesitaremos tener la base de datos ya creada e importarla desde PostgreSQL. Reutilizamos el contenedor creado en la sesión de prácticas, con todos los archivos .jar copiados (el extended, apoc y driver JDBC necesarios para la importación de datos desde la SGBD).
Realizamos las consultas para importar los datos directamente desde nuestra base de datos PostregreSQL con una conexión JDBC.

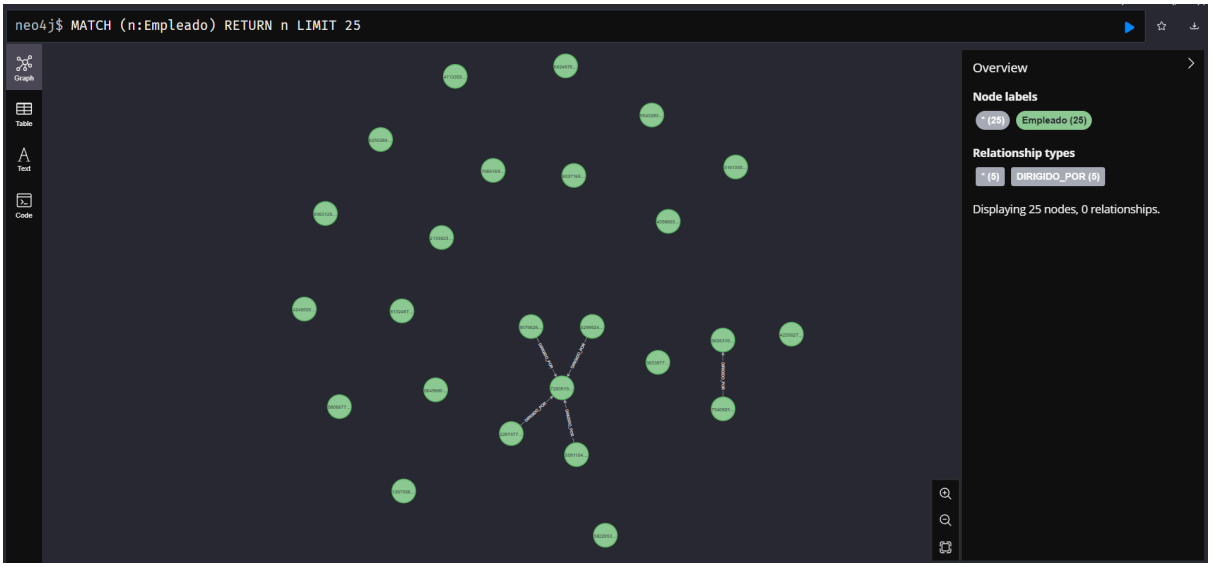
Para importar los nodos correctamente hay que tener en cuenta diversas cuestiones:

- ➔ Asegurarnos de que el contenedor de postgresql está en la misma red que el contenedor Neo4J con `docker network connect network-edge-gria edge-gria-pgsql`
- ➔ Primero debemos eliminar todos los nodos y arcos del grafo existentes con `match(n) with n limit 10000 detach delete n;`
- ➔ Para establecer la conexión debemos modificar el url de cada nodo con el nombre de nuestra base de datos, que en nuestro caso sería: `"jdbc:postgresql://edge-gria-pgsql/proyectohotel?user=alumnobd&password=pwalumnobd"`
- ➔ La clave primaria de Reserva es compuesta: (num_reserva, habitación, hotel) y en Neo4J no existen este tipo de claves primarias, por lo que debemos crear un identificador único combinando estos tres campos.
- ➔ Neo4J no acepta el formato geometry para los datos geográficos, por lo que hay que pasarlo a point en 2D. Cada Hotel y Punto de Interés necesita tener la localización como point().
  Para eso hay que extraer X/Y en el PostreSQL ejecutando los siguientes comandos en el DBeaver:

```
SELECT
 id,
 ST_X(ST_Centroid(localizacion)) AS x,
 ST_Y(ST_Centroid(localizacion)) AS y,
 nombre,
 fecha_inauguracion
        FROM hoteles.hoteles;
```

Una vez tenemos todo esto en cuenta, creamos los nodos de cada entidad con el código adjuntado en el archivo *nodos.txt.*

Ejemplo de grafo de entidad empleado con la relación dirigido_por:



- **Consulta 1:**

```
MATCH
(p:Persona)<-[:ES]-(c:Cliente)<-[:RESERVA_DE]-(r:Reserva)-[:PARA_HABITACION]->(h:Habitacion)
WHERE h.hotel_id = 1 AND h.piso = 5 AND date(r.fecha_entrada).year = 2025
RETURN p.nombre, p.dni, h.numero AS num_habitacion, r.duracion AS dias_alojamiento;
```

	p.nombre	p.dni	num_habitacion	dias_alojamiento
1	"Vicente Pérez"	"48783419F"	17	14
2	"Sergio Fernández"	"32212400K"	105	2
3	"Vicente Campos"	"56608488G"	204	2

- **Consulta 2:**

```
WITH 2 AS hotelId
MATCH (r:Reserva)-[:PARA_HABITACION]->(h:Habitacion)
WHERE h.hotel_id = hotelId
WITH date(r.fecha_entrada).year AS ano, sum(r.duracion * h.precio_noche) AS ingresos, hotelId
ORDER BY ano DESC
WITH collect({ano: ano, ingresos: ingresos}) AS ingresos_anuales, hotelId

UNWIND ingresos_anuales AS ia
OPTIONAL MATCH (e:Empleado)
WHERE e.hotel = hotelId AND date(e.fecha_incorporacion).year <= ia.ano
WITH ia.ano AS ano, ia.ingresos AS ingresos, sum(e.sueldo * 12) AS total_gastado
RETURN ano, ingresos, total_gastado
ORDER BY ano DESC;
```

	ano	ingresos	total_gastado
1	2025	62539.0	570312.0
2	2024	26754.0	570312.0
3	2023	38355.0	535380.0
4	2022	18024.0	500100.0
5	2021	22132.0	449160.0
6	2020	30546.0	408396.0
7			

- **Consulta 3:**

```
MATCH (p:Persona)<-[:ES]-(c:Cliente)<-[:REALIZADA_POR]-(r:Resena)
WHERE toLower(r.descripcion) CONTAINS 'baby'
RETURN p.nombre, r.valoracion, r.id_resena, r.descripcion
ORDER BY r.valoracion DESC;
```

	p.nombre	r.valoracion	r.id_resena	r.descripcion
Table	"Nicolás Méndez"	5	373	"superb not asked, stayed mayfair 6 days 1 bedroom suite perfect, traveling 10-month old baby looked internet nice hotel decent location separate bedroom accommodate baby sle
Text	"Clara Díaz"	5	28	"excellente resort resort excellent, went honeymoon just got, multiple restaurants choose, favorite lobster house chille garlic sauce grilled, bars great drank coconuts time coco-loco
Code	"David Blanco"	5	48	"wana, england baby, woop woop, stayed hotel late august early sept family, hotel beautiful staff amazing, met really good friends whilst staying hotel day wkd loads stuff going poo
	"Juan Sánchez"	5	260	"awesome family vacation just got family 4 kids 5 6 great time, brief overview.rooms plenty variety, easy switch need, n't unpack night look resort want, family room living room w/ 2
	"David Santos"	4	239	"loved place winter wife 1 year old child, loved place room better cuba santa clara resorts room okay, clean nice view, wasnt able visit al'carte restaurants baby buffet great, baby w
	"David Díaz"	1	269	"horrific despite friendly assistance time booking nothing prepared, booked quad room 2 adults 2 kids baby stay 7 nights, hotel website staff tell quad rooms alley hotel no photo we
	"Vicente Pérez"	1	626	"not worth price date stay november 16-19 2008my family arrived hotel conference dates listed, arrived decided not valet car parked self-parking lot, knew long walk self parking lot
	"Nicolás Ortega"	1	423	"not book hotel 5-6 months, arrived decribed building site, picked jaws ground asked builder going directed reception hard hats supplied place dangerously derelict downstairs hotel

- **Consulta 4:**

```
// Parte 1: Clientes con más de 5 reservas en un hotel
MATCH
(p:Persona)<-[:ES]-(c:Cliente)-[:RESERVA_DE]-(r:Reserva)-[:PARA_HABITACION]->(hab:Habitacion)-[:PERTENECE_A]->(h:Hotel)
WITH h.nombre AS hotel, p.nombre AS cliente, c.dni_cliente AS dni, count(r) AS total_reservas
WHERE total_reservas > 5
RETURN hotel, cliente, dni
```

```
UNION

// Parte 2: Clientes con más de 5 reseñas en un hotel
MATCH
(p:Persona)<-[:ES]-(c:Cliente)<-[:REALIZADA_POR]-(res:Resena)-[:RESEÑA_DE]->(h:Hotel
)
WITH h.nombre AS hotel, p.nombre AS cliente, c.dni_cliente AS dni, count(res) AS
total_resenas
WHERE total_resenas > 5
RETURN hotel, cliente, dni ;
```

hotel	cliente	dni
"Hotel Puerta del Camino"	"Rodrigo Blanco"	"27482426R"
"Hotel Puerta del Camino"	"Cristina Díaz"	"86706296A"
"Hotel Puerta del Camino"	"Juan Sánchez"	"77368169P"
"Hostal dos Reis Católicos"	"Iván Vega"	"94415651E"
"Hostal dos Reis Católicos"	"Jesús García"	"68675158D"
"Hotel Puerta del Camino"	"Alfonso Blanco"	"13246617X"
"Hostal dos Reis Católicos"	"Alejandra Soto"	"76184458D"

- **Consulta 5:** No existe una función equivalente a *ranking* en Neo4J, por lo que no se puede implementar esta consulta.

- **Consulta 6:**

```
MATCH (e:Empleado)
WHERE NOT EXISTS {
  (e)<-[:DIRIGIDO_POR]-(:Empleado)
}
// Usamos OPTIONAL MATCH porque el jefe principal podría no tener director.
OPTIONAL MATCH (e)-[:DIRIGIDO_POR]->(jefe:Empleado)

RETURN e.dni_empleado AS DNI_Empleado_Sin_Subordinados,
    e.cargo AS Cargo_Empleado,
    CASE
      WHEN jefe IS NULL THEN 'N/A (Director Principal)'
      ELSE jefe.dni_empleado
    END AS DNI_Jefe_Directo,
    jefe.cargo AS Cargo_Jefe_Directo
ORDER BY DNI_Empleado_Sin_Subordinados
```

	DNI_Empleado_Sin_Subordinados	Cargo_Empleado	DNI_Jefe_Directo	Cargo_Jefe_Directo
1	"00651254V"	"Recepcionista"	"56144992N"	"Jefe de Recepción"
2	"02772688J"	"Técnico de Mantenimiento"	"06263162D"	"Jefe de Mantenimiento"
3	"02841500V"	"Técnico de Mantenimiento"	"42292514H"	"Jefe de Mantenimiento"
4	"02999244L"	"Botones"	"72009195L"	"Jefe de Recepción"
5	"04391822Y"	"Técnico de Mantenimiento"	"42292514H"	"Jefe de Mantenimiento"
6	"06011941A"	"Recepcionista"	"77054112P"	"Jefe de Recepción"
7	"06382995S"	"Recepcionista"	"77054112P"	"Jefe de Recepción"
8	"10527053A"	"Limpiador/a"	"45988552N"	"Jefa de Limpieza"
9	"10904380S"	"Auxiliar de Lavandería"	"45988552N"	"Jefa de Limpieza"

- **Consulta 7:**

```
MATCH (h:Hotel {id_hotel: 1})
MATCH (pi:PuntoInteres)
WHERE
  pi.amenity = 'restaurant'
  AND pi.cuisine = 'regional'
  // Usa point.distance para el cálculo geodésico
  AND point.distance(h.localizacion, pi.geom) <= 1000

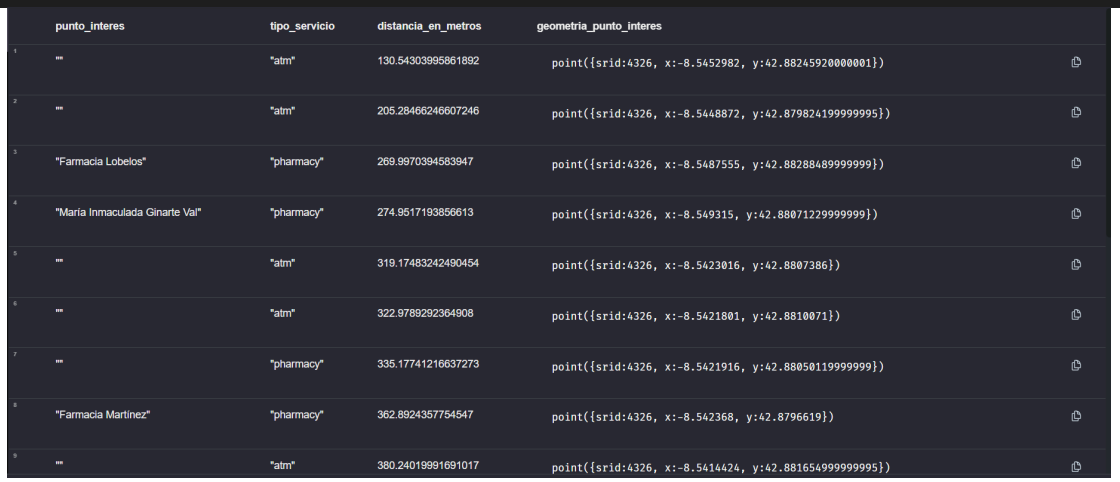

RETURN pi.nombre AS name,
       point.distance(h.localizacion, pi.geom) AS distancia_en_metros,
       pi.geom,
       h.localizacion
ORDER BY distancia_en_metros ASC;
```

	name	distancia_en_metros	pi.geom	h.localizacion
1	"El Pasaje"	282.5035205161576	point({srid:4326, x:-8.5456886, y:42.87755589999998})	point({srid:4326, x:-8.547303288180595, y:42.87531087665974})
2	"Casa Elisa"	336.4985593087208	point({srid:4326, x:-8.545444, y:42.87800919999999})	point({srid:4326, x:-8.547303288180595, y:42.87531087665974})
3	"Fogar de Galicia, Fogar do Santiso"	338.0094454319413	point({srid:4326, x:-8.5454541, y:42.878028099999995})	point({srid:4326, x:-8.547303288180595, y:42.87531087665974})
4	"El Papatorio"	379.6463538342625	point({srid:4326, x:-8.5453226, y:42.87839699999999})	point({srid:4326, x:-8.547303288180595, y:42.87531087665974})
5	"A Barrola"	381.0167739335888	point({srid:4326, x:-8.5451897, y:42.8783631})	point({srid:4326, x:-8.547303288180595, y:42.87531087665974})
6	"San Jaime"	522.3892936995279	point({srid:4326, x:-8.5447493, y:42.879614199999985})	point({srid:4326, x:-8.547303288180595, y:42.87531087665974})
7	"Artesana Gastrobar"	648.633804567335	point({srid:4326, x:-8.5528419, y:42.8711305})	point({srid:4326, x:-8.547303288180595, y:42.87531087665974})
8	"Casa Manolo"	785.6669137664018	point({srid:4326, x:-8.541991500000002, y:42.88119819999999})	point({srid:4326, x:-8.547303288180595, y:42.87531087665974})

- **Consulta 8:**

```
MATCH (h:Hotel {nombre: 'Hostal dos Reis Católicos'})
MATCH (pi:PuntoInteres)
WHERE pi.amenity IN ['atm', 'pharmacy']
  AND point.distance(h.localizacion, pi.geom) <= 500

RETURN pi.nombre AS punto_interes,
      pi.amenity AS tipo_servicio,
            point.distance(h.localizacion, pi.geom) AS distancia_en_metros, // Calculamos la distancia
      pi.geom AS geometria_punto_interes // El objeto POINT completo
ORDER BY distancia_en_metros ASC;
```



	punto_interes	tipo_servicio	distancia_en_metros	geometria_punto_interes
1	""	"atm"	130.54303995861892	point({srid:4326, x:-8.5452982, y:42.88245920000001})
2	""	"atm"	205.28466246607246	point({srid:4326, x:-8.5448872, y:42.879824199999995})
3	"Farmacia Lobelos"	"pharmacy"	269.9970394583947	point({srid:4326, x:-8.5487555, y:42.88288489999999})
4	"María Inmaculada Ginarte Vai"	"pharmacy"	274.9517193856613	point({srid:4326, x:-8.549315, y:42.88071229999999})
5	""	"atm"	319.17483242490454	point({srid:4326, x:-8.5423016, y:42.8807386})
6	""	"atm"	322.9789292364908	point({srid:4326, x:-8.5421801, y:42.8810071})
7	""	"pharmacy"	335.17741216637273	point({srid:4326, x:-8.5421916, y:42.88050119999999})
8	"Farmacia Martínez"	"pharmacy"	362.8924357754547	point({srid:4326, x:-8.542368, y:42.8796619})
9	""	"atm"	380.24019991691017	point({srid:4326, x:-8.5414424, y:42.881654999999995})

En Neo4J las relaciones se almacenan como grafos con punteros directos, lo que provoca que las consulta reflexivas se vean beneficiadas al permitir recorrerlas de forma mucho más eficiente y agilizada.

Al no requerir *joins*, ni *with recursive*, ni uniones repetidas, la complejidad temporal disminuye y el rendimiento mejora de manera significativa. En cambio, en SQL este tipo de consultas exige precisamente esas operaciones, lo que provoca una degradación del rendimiento a medida que aumenta la profundidad del recorrido y eleva el coste computacional tanto en CPU como en disco.

En el resto de consultas el rendimiento no varía, ya que no se navega el grafo, simplemente se hacen operaciones sobre datos tipo tabla, que en Neo4J están organizados igual que una SGBD relacional.

HBASE

En HBase las columnas se agrupan en familias de columnas o *column families* con el objetivo de optimizar las lecturas gracias a la co-localización física de los datos, se centra en las consultas tipo OLAP, permitiendo búsquedas y modificaciones en archivos grandes.

La clave de cada fila (Row Key) va a ser el id del hotel ya que las consultas suelen ser sobre un hotel en concreto y como familias de columnas creamos las especificadas en la tabla siguiente, la razón de tener varias es que los datos dentro de cada una se almacenan físicamente juntos y así podemos separar los que se consultan de formas diferentes o los que crecen más rápido de los estáticos.

<table>
<tr><th>CF</th><th>Descripción</th><th>Razón</th><th>Contenido</th></tr>
<tr><td>info</td><td>Datos básicos y estáticos del hotel.</td><td>Estos datos se consultan casi siempre juntos y no cambian. De esta forma se pueden devolver en una sola operación de lectura.</td><td>● nombre
● fecha_inauguración
● localización</td></tr>
<tr><td>habitaciones</td><td>Lista de habitaciones con el número de habitación como parte del nombre de la columna para diferenciarlas.</td><td rowspan="2">En estas CF se almacenan datos relacionados con los trabajadores y las habitaciones, separados de la información básica del hotel, así consultar esa información no fuerza a cargar todas las habitaciones o empleados.
El uso de número de habitación y DNI como parte del calificador de columna permite guardar cada entidad como un archivo único en la CF y así se optimizan las búsquedas por ese identificador.</td><td>● tipo
● precio
● capacidad
● piso</td></tr>
<tr><td>empleados</td><td>Plantilla de empleados con el dni como parte de la columna para diferenciarlos.</td><td>● nombre
● cargo
● sueldo</td></tr>
<tr><td>resenas</td><td>Información de las reseñas con el ID de reseña como parte de la columna para diferenciarla.</td><td rowspan="2">Estos son los datos históricos, los que más pueden aumentar en volumen y se usan para analitícas.
Cómo están separadas en dos CF se pueden escanear reseñas sin cargar las reservas y se pueden añadir nuevas reservas o reseñas fácilmente (PUT) porque las escrituras se ejecutan de forma atómica a nivel de fila. Además, al tener el ID reseña y número de reserva como parte del calificador permite guardar una lista infinita de reservas o reseñas que se pueden recuperar de forma muy rápida (SCAN).</td><td>● texto
● valoración
● nombre_cliente</td></tr>
<tr><td>reservas</td><td>Historial de reservas con el número de reserva como parte de la columna para diferenciarlas.</td><td>● fecha llegada
● cliente
● duración</td></tr>
</table>

Tiene una caché de bloques que permite optimizar el rendimiento al priorizar el acceso a familias de columnas más críticas, se puede configurar de forma que el contenido que se consulta mucho como las habitaciones, permanezcan en la RAM y otras tablas menos utilizadas como reseñas no se almacenen prioritariamente en la caché y se guarden en disco, almacenado ahí los datos históricos.

CONSULTAS

1. Ficha Técnica del Hotel (Consulta por ID)

Esta es la operación más "natural" en HBase. En lugar de hacer SELECT * FROM habitacion WHERE id_hotel = 1(SQL), aquí traes el **Objeto Hotel entero** con un solo acceso a disco.

- **Lo que hace:** Accede directamente a la fila "1". Recorre la familia de columnas habitaciones. Como las columnas son dinámicas (101_tipo, 102_tipo), el código detecta automáticamente cuántas habitaciones hay y muestra su tipo.
- **Utilidad:** Ideal para la pantalla de "Detalle de Hotel" en una web de reservas.

2. Buscador de Habitaciones Baratas (Scan con Filtro)

HBase no tiene un índice de precios. Para encontrar habitaciones baratas, hay que "abrir" todos los hoteles.

- **Lo que hace:** Abre un Scan (cursor) sobre la tabla. Para cada hotel, mira solo la familia habitaciones. Convierte el texto del precio a número (Double) y aplica un if (precio < 100).
- **Utilidad:** Permite responder preguntas de negocio sobre el inventario global, aunque el coste computacional lo asume tu código Java, no la base de datos.

3. Auditoría de Calidad (Correlación de Columnas)

Esta es interesante porque demuestra cómo relacionar dos columnas dinámicas que pertenecen al mismo concepto lógico.

- **Lo que hace:** Escanea la familia resenas. Busca columnas que terminen en _valor (la nota numérica). Si la nota es baja (menos de 3), extrae el ID de esa reseña (el prefijo) y busca automáticamente la columna hermana _textopara mostrar el motivo de la queja.
- **Utilidad:** Muestra cómo reconstruir objetos lógicos (una reseña completa) que fueron "aplanados" en columnas separadas durante la migración.

Antes de nada, para poder ejecutar las consultas en HBase tenemos que poner estos comandos en la terminal de WSL.

- **bin/hbase shell**
- **scan 'hoteles'**

Ahora ya podemos ejecutar el archivo que contiene las consultas a realizar:

- **Consulta 1:** Queremos saber qué habitaciones hay en el Hotel Araguaney.

```
System.out.println(">>> 1. FICHA TÉCNICA DEL HOTEL ID 1 <<<");

Get getHotel = new Get(Bytes.toBytes( s: "1")); // ID 1
Result r1 = table.get(getHotel);

if (!r1.isEmpty()) {
    // Recuperamos nombre (Fijo)
    byte[] bNombre = r1.getValue(Bytes.toBytes( s: "info"), Bytes.toBytes( s: "nombre"));
    System.out.println("HOTEL: " + Bytes.toString(bNombre));

    // Recuperamos habitaciones (Dinámico)
    System.out.println("  [Inventario de Habitaciones]:");
    for (Cell cell : r1.listCells()) {
        String familia = Bytes.toString(CellUtil.cloneFamily(cell));
        String columna = Bytes.toString(CellUtil.cloneQualifier(cell)); // ej: 101_tipo
        String valor = Bytes.toString(CellUtil.cloneValue(cell));

        if (familia.equals("habitaciones") && columna.endsWith("_tipo")) {
            // Truco visual: extraemos el número de la habitación del nombre de la columna
            String numHab = columna.split( regex: "_")[0];
            System.out.println("   - Hab " + numHab + ": " + valor);
        }
    }
} else {
    System.out.println("Hotel 1 no encontrado.");
}
```

```
>>> 1. FICHA TÉCNICA DEL HOTEL ID 1 <<<
HOTEL: Hotel Araguaney
  [Inventario de Habitaciones]:
   - Hab 100: Doble
   - Hab 101: Suite
   - Hab 102: Doble
   - Hab 103: Doble
   - Hab 104: Doble
   - Hab 105: Doble
   - Hab 106: Individual
   - Hab 107: Doble
   - Hab 108: Doble
   - Hab 109: Individual
   - Hab 10: Familiar
   - Hab 110: Doble
   - Hab 111: Doble
   - Hab 112: Doble
   - Hab 113: Doble
   - Hab 114: Doble
```

- **Consulta 2:** Un cliente nos pide buscar habitaciones que están en oferta. Su presupuesto máximo es de 100€.

```
System.out.println(">>> 2. BUSCADOR: Habitaciones por debajo de 100€ <<<");

Scan scanPrecios = new Scan();
scanPrecios.addFamily(Bytes.toBytes( s: "habitaciones")); // Optimización: Solo traer esta famili

ResultScanner scanner = table.getScanner(scanPrecios);

for (Result r : scanner) {
    String nombreHotel = "Hotel ID " + Bytes.toString(r.getRow()); // RowKey

    for (Cell cell : r.listCells()) {
        String columna = Bytes.toString(CellUtil.cloneQualifier(cell));
        String valor = Bytes.toString(CellUtil.cloneValue(cell));

        // Buscamos columnas de precio (ej: 101_precio)
        if (columna.endsWith("_precio")) {
            try {
                double precio = Double.parseDouble(valor);
                if (precio < 100.0) {
                    String numHab = columna.split( regex: "_")[0];
                    System.out.println("¡OFERTA! " + nombreHotel + " -> Hab " + numHab + " por "
                }
            } catch (Exception e) { /* Ignorar datos corruptos */ }
        }
    }
}
scanner.close();
```

```
>>> 2. BUSCADOR: Habitaciones por debajo de 100€ <<<
Hotel ID 1 -> Hab 106 por 65.0€
Hotel ID 1 -> Hab 109 por 65.0€
Hotel ID 1 -> Hab 119 por 70.0€
Hotel ID 1 -> Hab 141 por 79.0€
Hotel ID 1 -> Hab 145 por 91.0€
Hotel ID 1 -> Hab 155 por 86.0€
Hotel ID 1 -> Hab 160 por 91.0€
Hotel ID 1 -> Hab 171 por 77.0€
Hotel ID 1 -> Hab 172 por 75.0€
Hotel ID 1 -> Hab 174 por 70.0€
Hotel ID 1 -> Hab 179 por 79.0€
Hotel ID 1 -> Hab 17 por 76.0€
Hotel ID 1 -> Hab 181 por 77.0€
Hotel ID 1 -> Hab 183 por 74.0€
```

- **Consulta 3:** Queremos hacer un análisis de las reseñas que han dejado los clientes hasta el momento, por ello queremos seleccionar las reseñas con puntuación inferior a 3 para saber la opinión de eses clientes insatisfechos con el objetivo de mejorar.

```
System.out.println(">>> 3. ALERTA: Reseñas negativas (Valoración < 3) <<<");

Scan scanCalidad = new Scan();
scanCalidad.addFamily(Bytes.toBytes( s: "resenas"));

ResultScanner scannerCalidad = table.getScanner(scanCalidad);

for (Result r : scannerCalidad) {
    String idHotel = Bytes.toString(r.getRow());

    for (Cell cell : r.listCells()) {
        String columna = Bytes.toString(CellUtil.cloneQualifier(cell));
        String valor = Bytes.toString(CellUtil.cloneValue(cell));

        // Buscamos columnas de valoración (ej: 1_valor)
        if (columna.endsWith("_valor")) {
            try {
                int estrellas = Integer.parseInt(valor);
                if (estrellas < 3) {
                    // Hemos encontrado una queja. Ahora buscamos el texto asociado.
                    // El ID de la reseña es el prefijo (ej: "1" de "1_valor")
                    String idResena = columna.split( regex: "_")[0];

                    // Buscamos el texto en la misma fila (Result) sin hacer otra consulta
                    byte[] valTexto = r.getValue(Bytes.toBytes( s: "resenas"), Bytes.toBytes( s: idResena + "_texto"));
                    String textoQueja = (valTexto != null) ? Bytes.toString(valTexto) : "Sin texto";

                    System.out.println("Hotel " + idHotel + " (Nota: " + estrellas + "):");
                    System.out.println("   \"" + textoQueja + "\"");
                }
            } catch (Exception e) { }
```

```
>>> 3. ALERTA: Reseñas negativas (Valoración < 3) <<<
Hotel 1 (Nota: 1):
   "worst hotel, inside letter hotel, no reply april 16 2007casablanca ocean6345 collins avemiami beach fl 33141casablanca family wi
Hotel 1 (Nota: 2):
   "not good choice barcelona stayed princess sofia hotel business trip october 2005. hotel clean located stadium business section c
Hotel 1 (Nota: 1):
   "not stay thieves, stayed boyfriend 3 nights spending 10 days orlando 30th birthday, luckily fantastic time orlando, riu hotel di
Hotel 1 (Nota: 1):
   "bah service not impressed hotel rooms given supposedly group rate 129.00 courtyard rooms, n't courtyard room ran, party cancelle
Hotel 1 (Nota: 1):
   "hotel nightmare 2005- 2005 construction- awful mildew smell- service service, normandy hotel eerie feeling, husband broke toe go
Hotel 1 (Nota: 1):
   "hotel theft deception just got stay hotel langlois miserable experience feel share travelers, hotel room robbed stay hotel manag
Hotel 1 (Nota: 1):
   "bad service deeply disappointed service, promised specifichotel room arrived said did not types rooms hotel, confirmation sent m
Hotel 1 (Nota: 1):
   "yecch, hotel la playa seedy feel moment, walked room smell cigarette smoke overwhelming, surprised moment commenting maid emerge
Hotel 1 (Nota: 2):
   "gentle stay impression slightly dissapointing not glitsy website portrayed staff busy.however day staff attentive saw past ordin
Hotel 1 (Nota: 2):
   "not star hotel just returned week stay paradisus palma real, hotel beautiful problems work, did not opt royal service sure glad
Hotel 1 (Nota: 1):
```